

Tecnologia em Análise e Desenvolvimento de Sistemas

Setor de Educação Profissional e Tecnológica - SEPT

Universidade Federal do Paraná - UFPR

Prof. Alexander Robert Kutzke

DS122 - Desenvolvimento de Aplicações Web 1

Material da Disciplina DS122 - Desenvolvimento de Aplicações Web 1

Pré-prompt para uso durante as aulas

Sempre que iniciar uma nova conversa sobre a disciplina, cole o **Prompt de Configuração** (fornecido pelo professor) na primeira mensagem. Isso ajustará a IA para ser direta e técnica, removendo mensagens motivacionais e impedindo que ela entregue respostas prontas.

“Atue como um tutor socrático especializado em Ciência da Computação. Seu objetivo é auxiliar no meu desenvolvimento técnico e analítico, respeitando as seguintes diretrizes de interação:

1. **Objetividade Técnica:** Comunique-se de forma estritamente direta e profissional. Abstenha-se de mensagens de saudação, frases motivacionais, cortesias ou preâmbulos. Vá diretamente ao conteúdo técnico solicitado.
 2. **Restrição de Código-Solução:** Está terminantemente proibido fornecer implementações completas ou blocos de código que resolvam exercícios, problemas ou projetos que eu apresentar.
 3. **Exemplos Conceituais e Abstratos:** Caso seja necessário ilustrar um conceito ou sintaxe, forneça exemplos genéricos e minimalistas (máximo de 5 linhas). Estes exemplos não devem ter relação direta com o contexto do problema que estou tentando resolver.
 4. **Mediação Socrática:** Diante de dúvidas ou erros no meu código, não ofereça a correção. Em vez disso, aponte a inconsistência lógica ou o conceito de documentação relevante e formule perguntas que me conduzam à autocrítica e à resolução autônoma.
 5. **Foco em Arquitetura e Algoritmos:** Priorize a discussão sobre estruturas de dados, complexidade algorítmica e lógica de programação antes de abordar particularidades de sintaxe.
 6. **Verificação de Retenção (Opcional):** Apenas se eu incluir a hashtag **#validar** em minha pergunta, você deve, após a explicação, solicitar que eu descreva a lógica subjacente com minhas próprias palavras para validar minha compreensão.”
-

O Comando #validar

O item 6 do nosso protocolo é uma ferramenta de **Metacognição**. Você tem o controle sobre quando quer ser testado pela IA.

Quando utilizar o #validar ?

- Ao aprender um conceito novo e abstrato (ex: Ponteiros, Recursão, Herança).
- Quando a explicação da IA parecer clara, mas você não tem certeza se saberia aplicar sozinho.
- Para garantir que você não está apenas “copiando a lógica” da IA sem entendê-la.

Exemplo de uso no chat:

“Não entendi como funciona o desempenho em uma função recursiva. Pode me explicar?
#validar”

O que acontece depois:

A IA explicará o conceito de forma breve e, em seguida, pedirá que **você** explique a lógica de volta para ela. Ela só confirmará se você acertou após ler a sua explicação.

Melhores Práticas para Alunos de Engenharia/TI

O que FAZER

- Enviar o seu código com erro e perguntar: “Em qual conceito de lógica eu falhei aqui?”
- Pedir exemplos genéricos de sintaxe (ex: “Como declarar um vetor em C++?”).
- Usar a IA para interpretar mensagens de erro do compilador.

O que EVITAR

- Colar o enunciado do exercício e pedir: “Resolva para mim”.
 - Pedir que a IA escreva a função específica do seu trabalho acadêmico.
 - Aceitar uma solução da IA sem entender cada linha do que foi sugerido.
-

Lembre-se:

A IA no ensino superior deve ser uma ferramenta de apoio ao **aprendizado**. Se você apenas pedir respostas completas, estará diminuindo sua capacidade de resolver problemas reais e complexos.

Sobre Mob Programming para exercícios

Instruções: Mob Programming

O que é? O Mob Programming é uma técnica de desenvolvimento onde o grupo trabalha no mesmo código, ao mesmo tempo, utilizando apenas um computador. O objetivo é unificar o conhecimento e evitar que apenas um aluno resolva o problema.

Como funciona (Os Papéis)? A equipe atua com dois papéis estritos, que são alternados durante a aula:

1. **Piloto:** É a única pessoa com as mãos no teclado e mouse. O Piloto não codifica por conta própria; sua função é digitar estritamente o que o grupo instruir.
2. **Navegadores:** São os demais membros do grupo. Eles discutem o problema, pesquisam a documentação, formulam a lógica e ditam para o Piloto exatamente o que deve ser escrito.

Regras de Mob Programming nessa disciplina:

- **Formação:** Duplas ou trios.
- **Equipamento:** Apenas um computador deve ser utilizado por grupo para a escrita do código. Os demais podem usar dispositivos apenas para leitura de documentação.
- **Exercício:** O professor disponibilizará o exercício na UFPR Virtual ou material da disciplina.
- **Rotação (Timer):** Um cronômetro de **7 minutos** será projetado na tela. Este é o tempo ideal para manter o foco e garantir que todos programem sem fadiga.
- **A Troca:** Quando o alarme tocar, o Piloto solta o teclado imediatamente e torna-se um Navegador. O próximo aluno assume como Piloto. A troca deve ser rápida para não pausar o fluxo de raciocínio.

Sempre utilize o **Pré-prompt sugerido** quando for fazer uso de IA generativa nas aulas da disciplina.

Aula 01 - Apresentação

Plano de ensino e avaliação

Consultar moodle da disciplina (<https://ufprvirtual.ufpr.br>).

O que aprenderemos?

- Funcionamento do paradigma cliente-servidor;
- Web e internet;
- Protocolo HTTP (HyperText Transfer Protocol) que rege as interações entre serviços na Web;
- Linguagens:
 - HTML (HyperText Markup Language);
 - CSS (Cascading Style Sheets);
 - JS (JavaScript):
 - Biblioteca JQuery;
 - PHP.

Q. Essa disciplina é difícil?

Não. O conteúdo apresentado durante a disciplina é, em geral, simples. Além disso, é um tema interessante e extremamente requisitado nos dias atuais.

Entretanto, pela listagem acima, percebe-se que o conteúdo é vasto. São, pelo menos, 4 novas linguagens a serem aprendidas pelos alunos e pelas alunas.

Outros complicador merecem atenção dos alunos e das alunas: A disciplina está no segundo semestre do curso: nessa etapa, o contato dos alunos e das alunas com disciplinas de programação ainda está no início. Em oposição a isso, esta disciplina exige, a partir do segundo mês, uma prática mais intensiva de programação.

Enfim, embora a disciplina aparente ter conteúdos simples, o contexto em que ela se insere a torna um tanto trabalhosa. Nesse sentido, indico, desde o primeiro dia, que alunos e alunas se dediquem a essa disciplina, busquem conteúdo além das aulas, procurem o professor para tirar dúvidas e, acima de tudo, façam exercícios e pratiquem.

Definitivamente essa é uma das disciplinas com maior número de materiais de qualidade disponíveis na internet. Aproveitem e tirem o melhor deles! ;)

Sobre o ambiente de desenvolvimento

Nessa disciplina, não utilizaremos nenhum software específico para a produção de arquivos HTML, CSS, JS e PHP. Utilizaremos editores de texto de propósito geral. Cada aluno pode utilizar aquele que preferir. Sugerem-se os seguintes:

- [Vscod](#);
- [Sublime Text](#);
- [NeoVim](#) (caso goste de ambiente de terminal);
- Qualquer outro minimamente viável.

Além disso, no decorrer da disciplina, alguns conteúdos estarão disponíveis em repositórios GIT, como o próprio material que está lendo agora. Esses repositórios estão armazenados no serviço Gitlab.com (<https://gitlab.com/>). Para maiores informações leia [aqui](#).

Portanto, embora opcional, é indicado que cada aluno e cada aluna instale os softwares necessários para trabalhar com Git:

- [Git](#)

Outros serviços

Os computadores do laboratório, tanto no Windows quanto no Linux, possuem as ferramentas acima instaladas.

Embora o PHP esteja apto a ser executado em qualquer plataforma, a experiência de instalá-lo em um ambiente Unix produz conhecimentos interessantes para o aluno. Assim, sugere-se que o aluno ou a aluna que tenha interesse, instale uma máquina virtual Linux em seu próprio computador ou utilize o Ubuntu instalado nos computadores durante as aulas (principalmente na segunda metade da disciplina).

No decorrer das aulas, mais comentários serão feitos sobre essas instalações.

Existem, ainda, algumas outras opções limitadas disponíveis online, como:

- <https://codeanywhere.com>

Aula - Ambientação com o Git



O objetivo dessa aula é termos uma primeira experiência com o uso do git e do Gitlab. Para isso, faça o seguinte (seguindo os passos do professor em aula):

1. Realize o seu primeiro acesso no gitlab.com;
2. Crie o seu grupo para as tarefas da disciplina ([instruções](#));
3. Resolva a [primeira tarefa](#).

Aula - Protocolo HTTP

Bibliografia recomendada para o tema:

- Cap 7.3 do livro do Tanenbaum (ver plano de ensino no moodle);
- [MDN - HTTP](#).

Modelo Cliente-Servidor

Diferentemente do modelo de programação para desktop (em geral, com um único ator), o modelo de programação *cliente-servidor* conta com dois atores:

- **Cliente:** Realiza **requisições**, uma a uma, e aguarda respostas;
- **Servidor:** Processa requisições e as **responde**, uma a uma.

Protocolo HTTP

Na web, o protocolo que gerencia a troca de informações (requisições e respostas) entre clientes e servidores é o HTTP (HyperText Transfer Protocol).

Segundo a Wikipedia¹:

O Hypertext Transfer Protocol (HTTP), em português Protocolo de Transferência de Hipertexto, é um protocolo de comunicação (na camada de aplicação segundo o Modelo OSI) utilizado para sistemas de informação de hipermídia, distribuídos e colaborativos. Ele é a base para a comunicação de dados da World Wide Web.

Hipertexto é o texto estruturado que utiliza ligações lógicas (hiperlinks) entre nós contendo texto. O HTTP é o protocolo para a troca ou transferência de hipertexto.

Coordenado pela World Wide Web Consortium e a Internet Engineering Task Force, culminou na publicação de uma série de Requests for Comments; mais notavelmente o RFC 2616, de junho de 1999, que definiu o HTTP/1.1. Em Junho de 2014 foram publicados 6 RFC's para maior clareza do protocolo HTTP/1.1. Em Março de 2015, foi divulgado o lançamento do HTTP/2.

RFC's (*Requests for comments*) são a forma oficial de como os protocolos, regras, especificações e padrões de funcionamento da web são divulgados.

O criador do protocolo HTTP foi Tim Berners-Lee.

Funcionamento do Protocolo HTTP



Uniform Resource Locator (URL)

URL's são a forma utilizada para se localizar serviços na Web. Trata-se de uma sequência de caracteres, com estrutura definida, capaz de identificar onde e como um dado serviço será acessado.

Para mais detalhes e exemplo, consulte: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Identifying_resources_on_the_Web

Interessado? Pesquise sobre a diferença entre URL, URN e URI (pode começar por [aqui](#))

Exemplo de mensagem HTTP

Uma requisição:

```
GET /index.html HTTP/1.1  
Host: www.example.com
```

Uma resposta:

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Encoding: UTF-8
Content-Length: 138
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close
```

```
<html>
<head>
  <title>An Example Page</title>
</head>
<body>
  Hello World, this is a very simple HTML document.
</body>
</html>
```

Toda mensagem HTTP recebe um código como resposta. Para uma breve lista deles, consulte: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

O protocolo possui diferentes métodos, sendo os mais conhecidos:

- GET: parâmetros enviados diretamente pela URL;
- POST: parâmetros enviados no corpo da mensagem.

Sobre todos os métodos e parâmetros possíveis em um cabeçalho HTTP, consulte: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>

Para realizar um teste de uma mensagem HTTP, é possível utilizar os comandos `telnet` e `nc` (netcat). A seguir, um exemplo com `nc` é descrito.

Em ambientes Unix e GNU/Linux, é provável que o comando `nc` já esteja instalado.

Em sistemas Windows, é possível utilizar a maquina virtual com vagrant ou instalar o comando `nc`.

Instalando o nc no Windows

Baixe o seguinte arquivo:

<https://joncraton.org/files/nc111nt.zip>

Descompacte o arquivo `nc.exe` em uma pasta de fácil acesso. **A senha para descompactar é “nc”.**

Agora, basta acessar pelo terminal (gitbash) a pasta onde o arquivo foi descompactado e utilizar o comando `nc` para executar o programa.

Exemplo de mensagem HTTP com nc

Para simular uma mensagem HTTP, é preciso informar ao `nc` qual o endereço queremos acessar (do servidor) e qual a porta devemos tentar conexão. Em geral, servidores de página WEB *escutam* na porta 80.

```
nc example.com 80
```

O comando ficará em modo de espera para digitarmos a mensagem a ser enviada. Se preferir, copie e cole o exemplo abaixo:

```
GET / HTTP/1.1
Host: example.com
```

Como cabeçalho HTTP deve terminar com uma linha vazia, lembre-se de pressionar `Enter` duas vezes. Para sair do comando `nc`, pressione `CTRL-D`.

Se desejar, é possível salvar a mensagem em um arquivo, digamos `teste.http` e passar o conteúdo do arquivo diretamente ao `nc` com o seguinte comando:

```
cat teste.http - | nc example.com 80
```

Caraterísticas importantes do HTTP

- Protocolo sem estado (*state-less*);
- Conexões persistentes a partir do HTTP 1.1;
- Possibilidade do uso de *Cookies*:
 - Enviados no cabeçalho da mensagem;
 - Servidor solicita o salvamento de pequenos conjuntos chaveXvalor no cliente;
 - Permeia questões de segurança.
- HTTPS:
 - TLS/SSL;
 - Mensagens criptografadas;
 - Certificação do cliente e do servidor.

Ferramentas de desenvolvimento no seu navegador

Qualquer navegador Web decente, disponibiliza ferramentas de desenvolvimento. Com elas, é possível explorar todos os detalhes envolvidos nas páginas exibidas pelo navegador, como: HTML, CSS, Javascript, mensagens HTTP trocadas, console de comandos, etc.

Explore as ferramentas no seu navegador. Em geral, o atalho `CTRL+SHIFT+I` ativa tais ferramentas.

O uso dos navegadores Internet Explorer e Microsoft Edge não são indicados para essa disciplina.

Navegadores indicados:

- Mozilla Firefox;
- Chromium e afins;
- Opera.

Prática

Faça requisições HTTP para diferentes sites e analise as respostas obtidas. Utilize algumas das ferramentas abaixo:

- Seu navegador;
- Comandos como o nc;
- Extensões para navegadores;
- <https://httpie.org/run>

¹ Tenha sempre em mente que são raras as vezes em que a Wikipedia pode ser utilizada como uma referência científica relevante.

Aula - HTML5: Estrutura Semântica e Acessibilidade

Bibliografia Recomendada

- [MDN Web Docs - HTML](#)
 - [W3C - HTML5 Semantic Elements](#)
-

HyperText Markup Language

O HTML (HyperText Markup Language), é, como o nome diz, uma linguagem de marcação para criação de documento hipertexto.

Estrutura Base e Declaração de Idioma

Diferente das versões anteriores, o HTML5 simplificou o `doctype`. É obrigatório definir o atributo `lang` na tag `html` para acessibilidade (leitores de tela) e SEO.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Título do Documento</title>
</head>
<body>

</body>
</html>
```

Tags e atributos

A linguagem conta com um conjunto de palavras-chave denominadas `tags`. De maneira geral, uma `tag` possui o seguinte formato:

```
<nome_tag atributo1="valor1" atributo2="valor2" ...>conteúdo</nome_tag>
```

Onde `<nome_tag ...>` é chamada de tag de abertura e `</nome_tag>` é denominada tag de fechamento. Vale ressaltar que nem todas as tags possuem fechamento, nesse caso, a sintaxe é a seguinte:

```
<nome_tag atributo1="valor1" atributo2="valor2" ... />
```

Cada tag pode receber zero ou mais **atributos**. Um atributo indica características referentes a tag em questão. Por exemplo:

```
<a href="http://www.tads.ufpr.br">Moodle do TADS</a>
```

A tag acima irá produzir um link (ou âncora) para a url `http://www.tads.ufpr.br`, sendo exibida na página com o texto `Moodle do TADS`. Algo semelhante a isso: [Moodle do TADS](#).

Isso ocorre pois a tag `a` indica a criação de um link e, por meio do parâmetro `href`, o navegador sabe para qual URL o link deve levar ao ser clicado.

Características do HTML

O HTML deve ser utilizado para especificar apenas duas coisas:

- O **conteúdo** do documento (títulos, parágrafos, figuras, etc.);
- A **estrutura** do documento (isso é um título, aquilo é um link).

Ou seja, o HTML deve ser utilizado para definir a **semântica** do seu conteúdo, seu significado. **Não se deve utilizar o HTML para indicar nenhum tipo de formatação ou posicionamento do conteúdo.**

Sintaxe e Boas Práticas

- **Case Sensitivity:** Embora o navegador aceite maiúsculas, o padrão de mercado e as normas da W3C exigem tags e atributos em **minúsculo**.
- **Aspas:** Atributos devem ser envolvidos por aspas duplas (`" "`).
- **Self-closing tags:** No HTML5, a barra em tags vazias (ex: `<br />`) é opcional, mas o uso de `<br>` ou `<img>` é preferível por simplicidade.
- **Ferramental:** Utilize editores de texto para produzir seus arquivos HTML. O **VS Code** pode ser uma boa opção, mas você pode utilizar qual preferir.

HTML5

O HTML5 é a quinta e atual versão principal da HyperText Markup Language, o padrão da W3C para a estruturação de conteúdos na Web. Diferente de suas versões anteriores, o HTML5 evoluiu de uma simples linguagem de marcação para uma plataforma de desenvolvimento completa, introduzindo elementos semânticos (como `<article>`, `<section>` e `<nav>`), suporte nativo a multimídia (tags `<video>` e `<audio>`), e APIs poderosas para armazenamento local e geolocalização. Sua arquitetura foi projetada para reduzir a dependência de plugins proprietários e garantir que as aplicações web funcionem de forma consistente e responsiva em diversos dispositivos e navegadores.

Práticas modernas de Semântica e Estruturação do DOM

As principais tags introduzidas no **HTML5** que o diferenciam das versões anteriores (como o HTML 4.01) focam em substituir o uso genérico de `<div>` por elementos que descrevem a função do conteúdo para navegadores e tecnologias assistivas.

Tags de Estrutura Semântica

- **<header>** : Define o cabeçalho de uma página ou seção, geralmente contendo logotipos e navegação.
- **<nav>** : Identifica blocos que contêm links de navegação.
- **<main>** : Delimita o conteúdo principal e exclusivo do documento.
- **<article>** : Representa uma composição autônoma (post, comentário, notícia).
- **<section>** : Agrupa conteúdos logicamente relacionados em um mesmo tema.
- **<aside>** : Define conteúdos tangenciais, como barras laterais ou anúncios.
- **<footer>** : Define o rodapé de uma página ou seção.

Tags de Mídia e Gráficos

- **<video>** : Permite a reprodução de arquivos de vídeo nativamente, sem necessidade de plugins como o Flash.
- **<audio>** : Proporciona suporte nativo para execução de arquivos sonoros.
- **<canvas>** : Define uma área para renderização de gráficos e animações dinâmicas via JavaScript.
- **<svg>** : Permite a incorporação direta de gráficos vetoriais escaláveis no código HTML.

Tags de Organização de Conteúdo e Formulários

- **<figure>** e **<figcaption>** : Agrupam uma mídia (imagem, gráfico) e sua respectiva legenda.
 - **<mark>** : Utilizada para destacar ou realçar trechos de texto.
 - **<time>** : Representa datas e horários em um formato legível por máquinas.
 - **<datalist>** : Define uma lista de opções pré-definidas para controles de entrada de formulário.
-

Tags de Conteúdo e Acessibilidade

Imagens e Atributo alt

O atributo `alt` é obrigatório para acessibilidade. Descreve a imagem para deficientes visuais e mecanismos de busca.

```

```

Links (Âncoras)

Para links externos, utilize `rel="external"` ou `target="_blank"`.

```
<a href="https://www.tads.ufpr.br" target="_blank" rel="noopener">Portal TADS</a>
```

Formulários e Validação Nativa

O HTML5 introduziu tipos de input que realizam validação no lado do cliente sem necessidade de JavaScript inicial.

Exemplo de Formulário Técnico

```
<form action="processar.php" method="POST">
  <fieldset>
    <legend>Dados Pessoais</legend>

    <label for="nome">Nome Completo:</label>
    <input type="text" id="nome" name="nome" required placeholder="Digite seu
nome">

    <label for="email">E-mail:</label>
    <input type="email" id="email" name="email" required>

    <label for="nascimento">Data de Nascimento:</label>
    <input type="date" id="nascimento" name="nascimento">
  </fieldset>

  <fieldset>
    <legend>Preferências</legend>

    <label>Gênero:</label>
    <input type="radio" id="m" name="genero" value="M"> <label
for="m">Masculino</label>
    <input type="radio" id="f" name="genero" value="F"> <label
for="f">Feminino</label>

    <label for="veiculo">Veículo:</label>
    <select id="veiculo" name="veiculo">
      <option value="">Selecione...</option>
      <option value="carro">Carro</option>
      <option value="moto">Moto</option>
    </select>
  </fieldset>

  <button type="submit">Enviar Dados</button>
</form>
```

Principais Atributos de Validação

- `required` : Torna o campo obrigatório.
- `pattern` : Define uma Expressão Regular (Regex) para validação.

- `min / max` : Define limites numéricos ou de data.
 - `step` : Define o intervalo de incremento para números.
-

Elementos de Bloco vs. Em Linha (Inline)

- **Block-level:** Ocupam toda a largura disponível e começam em uma nova linha. Exemplos: `<div>`, `<h1>`, `<p>`, `<ul>`, `<li>`, `<header>`.
 - **Inline:** Ocupam apenas a largura necessária e não quebram linha. Exemplos: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`.
-

Exercícios Práticos: Estrutura e Semântica HTML5

Os exercícios abaixo devem ser realizados no **VS Code**, seguindo a indentação padrão (2 ou 4 espaços) e as normas de acessibilidade discutidas.

Exercício 01: Esqueleto Semântico

Crie um arquivo chamado `index.html` que represente a estrutura de um portal de notícias. O documento deve conter:

1. **Header:** Contendo um `<h1>` com o nome do portal e um `<nav>` com uma lista não ordenada (`<ul>`) de categorias (Home, Brasil, Tecnologia, Contato).
 2. **Main:** Contendo dois elementos `<article>`.
 - Cada `article` deve ter um `<h2>` (título da notícia) e um `<p>` (resumo).
 3. **Aside:** Uma barra lateral com links para "Postagens mais lidas".
 4. **Footer:** Informações de copyright e endereço.
-

Exercício 02: Formulário de Cadastro de Produtos

Desenvolva um formulário para um sistema de estoque (`cadastro_produto.html`). Utilize as tags de formulário e validações nativas:

- **Nome do Produto:** Campo de texto obrigatório.
- **Código SKU:** Campo de texto que aceite apenas o padrão de 3 letras seguidas de 4 números (Dica: use o atributo `pattern="[A-Z]{3}[0-9]{4}"`).
- **Quantidade em Estoque:** Campo numérico (`number`) com valor mínimo de 0 e máximo de 100.
- **Categoria:** Um componente `<select>` com as opções: Eletrônicos, Móveis e Alimentos.
- **Descrição:** Um `<textarea>` com limite de 200 caracteres.
- **Data de Recebimento:** Campo do tipo `date`.
- **Botões:** Um botão para "Limpar" (`type="reset"`) e outro para "Cadastrar" (`type="submit"`).

Exercício 03: Tabela de Dados Técnicos

Crie uma página que exiba uma tabela de especificações de hardware. A tabela deve seguir a estrutura semântica correta:

- Utilize as tags `<thead>`, `<tbody>` e `<tfoot>`.
 - Colunas: Componente, Especificação e Preço Médio.
 - Inclua pelo menos 3 linhas de dados (ex: Processador, Memória RAM, SSD).
 - No `<tfoot>`, utilize o atributo `colspan` para criar uma célula que ocupe as duas primeiras colunas com o texto "Total Estimado" e exiba o valor na terceira coluna.
-

Exercício 04: Acessibilidade e Mídia

Implemente uma página de galeria simples:

1. Insira uma imagem de um componente de hardware.
 2. Garanta que o atributo `alt` descreva a imagem tecnicamente.
 3. Utilize a tag `<figure>` para envolver a imagem e a tag `<figcaption>` para adicionar uma legenda visível abaixo dela.
 4. Crie um link que, ao ser clicado, faça o download de um manual em PDF (Dica: utilize o atributo `download` na tag `<a>`).
-

Validação

Para validar seus exercícios, utilize o [W3C Markup Validation Service](#). O código não deve apresentar erros ou avisos (warnings).

Aula CSS3: Fundamentos, Layout e Responsividade

Bibliografia Recomendada

- [MDN - CSS: Cascading Style Sheets](#)
- [CSS-Tricks - A Complete Guide to Flexbox](#)
- [CSS-Tricks - A Complete Guide to Grid](#)

Cascading Style Sheets (CSS3)

O CSS (Cascading Style Sheets) é a linguagem de folha de estilo utilizada para descrever a apresentação de um documento escrito em HTML. Sua principal função no desenvolvimento web é garantir a estrita separação entre a estrutura/semântica (HTML) e a apresentação visual (CSS).

Formas de Adicionar CSS ao HTML

Existem três métodos de integração, sendo o uso de arquivos externos o único recomendado para sistemas em produção:

1. **Atributo Style (Inline):** Aplica o estilo diretamente na tag. Tem a maior especificidade, mas fere o princípio de separação de responsabilidades. Não recomendado.

```
<p style="color: red; text-align: center;">Texto do parágrafo</p>
```

2. **Tag <style> (Interno):** Inserido no <head> do documento HTML.

```
<head>
  <style>
    p { color: red; }
  </style>
</head>
```

3. **Arquivo Externo (Recomendado):** Importação de um arquivo .css via tag <link> no <head> .

```
<head>
  <link rel="stylesheet" href="estilo.css">
</head>
```

Sintaxe e Seletores

A sintaxe baseia-se em uma declaração composta por uma **propriedade** e um **valor**, encapsulada em um bloco vinculado a um **seletor**.

```
/* Seletor de Elemento (Tag) */
h1 {
    color: white;
    text-align: center;
}

/* Seletor de Classe (.) - Reutilizável em múltiplos elementos */
.alerta {
    color: red;
    font-weight: bold;
}

/* Seletor de ID (#) - Deve ser único na página */
#menu-principal {
    background-color: #333;
}

/* Pseudo-classes (Estado do elemento) */
a:hover {
    text-decoration: underline;
}
```

Cascata e Especificidade

O termo “Cascading” refere-se ao algoritmo que o navegador utiliza para determinar qual regra CSS será aplicada quando há conflitos.

A resolução de conflitos segue a hierarquia de **Especificidade**:

1. **Inline style**: Maior peso.
2. **IDs**: Peso alto (ex: `#cabecalho`).
3. **Classes, atributos e pseudo-classes**: Peso médio (ex: `.card`, `[type="text"]`, `:hover`).
4. **Elementos e pseudo-elementos**: Menor peso (ex: `h1`, `div`, `::before`).

Se duas regras tiverem a mesma especificidade, a que foi declarada **por último** no código (cascata) prevalece.

Box Model (Modelo de Caixa) e Unidades de Medida

No CSS, todo elemento HTML é renderizado como uma caixa retangular.

- **Content**: A área onde o texto ou imagem é exibido.
- **Padding**: Espaçamento interno (entre o conteúdo e a borda).
- **Border**: A borda que envolve o padding e o conteúdo.
- **Margin**: Espaçamento externo (afasta o elemento dos elementos adjacentes).

Boa prática: Utilize a propriedade `box-sizing: border-box;` no reset global para garantir que o cálculo da largura (`width`) e altura (`height`) inclua o `padding` e a `border` , evitando quebras de layout.

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Unidades de Medida Modernas

O uso exclusivo de pixels (`px`) é desencorajado em layouts modernos.

- **rem** : Relativo ao tamanho da fonte do elemento raiz (`<html>`). Ideal para fontes e margens, pois respeita a configuração de acessibilidade do navegador do usuário.
 - **%** : Percentual em relação ao elemento pai.
 - ****vw / vh**** : Largura (`viewport width`) e altura (`viewport height`) relativas à janela do navegador.
-

Exercícios Práticos: Fundamentos de CSS3 (Pré-Layout)

Criar um arquivo `index.html` e um arquivo `style.css` na mesma pasta, realizando a vinculação através da tag `<link>` .

Exercício 01: Teste de Especificidade e Cascata

Objetivo: Compreender a hierarquia de aplicação de estilos e o cálculo de especificidade do navegador.

HTML base (`index.html`):

```
<div class="artigo" id="noticia-principal">  
  <h2 class="titulo">Lançamento do Novo Sistema</h2>  
  <p class="texto destaque">O sistema entrará em produção na próxima semana.</p>  
</div>
```

Requisitos (CSS):

1. Crie uma regra utilizando o seletor de tag `p` e defina a cor do texto como azul.
2. Crie uma regra utilizando o seletor de classe `.texto` e defina a cor do texto como verde.
3. Crie uma regra utilizando o seletor de classe `.destaque` e defina a cor do texto como laranja.
4. Crie uma regra encadeada `.artigo p.destaque` e defina a cor como roxo.
5. Adicione um estilo *inline* diretamente na tag `<p>` no HTML definindo a cor como vermelho.

Exercício 02: Tipografia, Cores e Unidades Relativas (rem)

Objetivo: Aplicar fontes externas e utilizar a unidade de medida `rem` atrelada ao elemento raiz.

Requisitos (CSS):

1. Importe a fonte "Roboto" ou "Open Sans" do Google Fonts via `@import` no início do arquivo CSS.
2. Defina no seletor `html` o `font-size: 16px;`.
3. Defina a fonte importada para o seletor `body` e aplique uma cor de fundo utilizando a notação Hexadecimal (ex: `#f4f4f9`).
4. Estilize um elemento `<h1>`: defina o tamanho da fonte para `2.5rem` e a cor do texto utilizando a notação `rgba()` com 80% de opacidade.
5. Estilize um elemento `<p>`: defina o tamanho para `1rem`, o espaçamento entre linhas (`line-height`) para `1.5` e o alinhamento de texto como justificado.

Exercício 03: O Box Model e box-sizing

Objetivo: Visualizar o impacto do `padding` e `border` no tamanho total de um elemento e corrigir o comportamento padrão com `box-sizing`.

HTML base:

```
<div class="caixa padrao">Caixa 1: Content-Box</div>
<div class="caixa corrigida">Caixa 2: Border-Box</div>
```

Requisitos (CSS):

1. Para a classe `.caixa`, defina `width: 300px`, `height: 150px`, cor de fundo amarela e `margin-bottom: 20px`.
2. Para ambas as caixas, adicione um `padding` de `30px` e uma `border` sólida preta de `5px`.
3. Na classe `.corrigida`, adicione a propriedade `box-sizing: border-box;`.

Exercício 04: Display e Centralização de Bloco

Objetivo: Manipular os valores da propriedade `display` (`block`, `inline`, `inline-block`) e aplicar centralização horizontal com margens automáticas.

HTML base:

```
<section class="container">
  <a href="#" class="botao">Clique Aqui</a>
  <a href="#" class="botao">Saiba Mais</a>
</section>
```

Requisitos (CSS):

1. Para a classe `.container`, defina `width: 50%`, uma cor de fundo cinza clara e aplique `margin: 0 auto;` para centralizar a seção horizontalmente na página.

2. A tag `<a>` por padrão é `inline`. Altere a classe `.botao` para `display: inline-block;`.

3. Defina para os botões:

- `padding: 10px 20px;`
- `margin-right: 15px;`
- Cor de fundo azul, texto branco e remova o sublinhado (`text-decoration`).

4. Adicione uma pseudo-classe `:hover` aos botões para alterar a cor de fundo para um azul mais escuro quando o mouse passar sobre eles.

Layouts Modernos: Flexbox e CSS Grid

Modelos de layout atuais substituem o uso de `float` e tabelas para posicionamento.

1. Flexbox (Layout Unidimensional)

Projetado para alinhar itens em uma única dimensão (linha ou coluna) e distribuir o espaço dinamicamente.

```
.flex-container {
  display: flex;
  flex-direction: row;           /* Organiza em linha */
  justify-content: space-between; /* Espaçamento horizontal */
  align-items: center;           /* Alinhamento vertical cruzado */
  gap: 15px;                     /* Espaçamento entre os filhos (CSS moderno) */
}
```

2. CSS Grid (Layout Bidimensional)

Projetado para organizar elementos simultaneamente em linhas e colunas.

```
.grid-container {
  display: grid;
  /* Cria 3 colunas: as das extremidades com 1 fração de espaço e a central com 2 */
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: auto 200px;
  gap: 20px;
}
```

Design Responsivo (Media Queries)

A responsividade garante que a interface se adapte a diferentes tamanhos de tela. O padrão de mercado é a abordagem **Mobile-first**: o CSS base é escrito para telas pequenas, e media queries são utilizadas para adaptar o layout para telas maiores.

```
/* Estilo base (Mobile - telas pequenas) */  
.coluna {  
    width: 100%;  
}  
  
/* Breakpoint para Tablets e Desktops (mínimo de 768px de largura) */  
@media screen and (min-width: 768px) {  
    .coluna {  
        width: 50%; /* Passa a ocupar metade da tela */  
    }  
}
```


Aula 06 - Programação Cliente (JavaScript Moderno)

Bibliografia recomendada

- [MDN Web Docs - JavaScript](#)
 - [w3schools - JavaScript Tutorial](#)
 - [Eloquent JavaScript](#) - Livro de referência sobre a linguagem.
-

JavaScript e ECMAScript

JavaScript é uma linguagem de programação interpretada, de tipagem dinâmica fraca e multiparadigma. Originalmente projetada para manipulação de páginas web no lado do cliente (navegador), sua aplicação foi expandida para o lado do servidor com o surgimento do **Node.js** e outros runtimes (como Deno e Bun).

Versões da Linguagem (O Padrão ECMAScript)

A linguagem JavaScript é padronizada pela organização ECMA International sob a especificação chamada **ECMAScript (ES)**.

- **ES5 (2009):** Versão legada, suportada por navegadores antigos.
- **ES6 ou ES2015:** O marco de modernização da linguagem. Introduziu uma nova sintaxe e recursos fundamentais como `let`, `const`, Arrow Functions, Classes, Promises e Template Strings.
- **ES7+ (Atualizações anuais):** A partir de 2015, o comitê adotou um ciclo de lançamento anual (ES2016, ES2017, etc.), adicionando melhorias incrementais.

No desenvolvimento atual, utiliza-se a sintaxe moderna (ES6+).

Utilizando JS em páginas Web

A inclusão de código JavaScript no HTML ocorre através da tag `<script>`. A prática moderna recomenda o uso de arquivos externos com o atributo `defer`.

Arquivo Externo com defer (Recomendado)

O atributo `defer` instrui o navegador a carregar o script em paralelo com o HTML, mas sua execução só ocorre após a renderização completa do DOM. Isso elimina a necessidade de colocar a tag `<script>` no final do `<body>`.

Arquivo HTML (index.html):

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Exemplo JS</title>
  <script src="app.js" defer></script>
</head>
<body>
  <h1>Página Web</h1>
  <p id="demo">Um parágrafo.</p>
  <button id="btnAcao">Clique aqui</button>
</body>
</html>
```

Arquivo JS (app.js):

```
// Boas práticas recomendam separar o comportamento (JS) da estrutura (HTML).
// Evite atributos como onclick no HTML.

document.getElementById('btnAcao').addEventListener('click', () => {
  document.getElementById('demo').innerHTML = "Parágrafo alterado.";
});

console.log("Arquivo carregado e DOM pronto!");
```

Variáveis e Escopo (ES6+)

No JavaScript moderno, o uso da palavra-chave `var` é **obsoleto** devido a problemas com o escopo de função e “hoisting” (elevação). Utilize estritamente `let` e `const`, que respeitam o **escopo de bloco**.

- **const**: Declara uma constante. O valor (ou a referência de memória para objetos/arrays) não pode ser reatribuído. Padrão preferencial.
- **let**: Declara uma variável que pode sofrer reatribuição.

```
// Escopo de Bloco
let x = "escopo global";

if (true) {
  let x = "escopo de bloco (if)";
  const y = "constante local";
  console.log(x); // → escopo de bloco (if)
}

console.log(x); // → escopo global
// console.log(y); // Erro de referência: y não está definido
```

Nota: A declaração implícita de variáveis (sem `let` ou `const`) cria variáveis globais não intencionais, o que é um erro em escopos modernos (especialmente usando `"use strict";`).

Tipos de Dados

O JavaScript possui tipagem dinâmica. A mesma variável pode armazenar diferentes tipos de dados em momentos distintos de sua execução.

Tipos Primitivos:

- `String`: Textos.
- `Number`: Números (inteiros e de ponto flutuante, incluindo `NaN`).
- `Boolean`: `true` OU `false`.
- `Undefined`: Variável declarada, mas sem valor atribuído.
- `Null`: Ausência intencional de valor.
- `Symbol`: Identificador único (ES6).
- `BigInt`: Números inteiros além do limite de precisão do `Number`.

Tipos Estruturais (Por referência):

- `Object`: Estruturas de chave/valor, Arrays, Datas.
- `Function`: Um subtipo de objeto invocável.

```
typeof "João";      // "string"
typeof 3.14;        // "number"
typeof false;       // "boolean"
typeof undefined;   // "undefined"
typeof null;        // "object" (Bug??)
typeof [1, 2, 3];    // "object" (Arrays são objetos no JS)
```

Strings Modernas: Template Literals (Template Strings)

No ES6+, utiliza-se crases (```) para manipulação de strings. Isso permite interpolação de variáveis e expressões diretamente na string, além de quebras de linha nativas.

```
const nome = "Maria";
const idade = 25;

// Concatenação legada
console.log("O nome dela é " + nome + " e ela tem " + idade + " anos.");

// Interpolação moderna (Template String)
console.log(`O nome dela é ${nome} e ela tem ${idade} anos.`);

// Expressões matemáticas dentro de strings
console.log(`Daqui a 5 anos, ela terá ${idade + 5} anos.`);
```

Funções

Funções em JavaScript são “cidadãs de primeira classe”, podendo ser armazenadas em variáveis, passadas como argumentos ou retornadas por outras funções.

1. Declaração Tradicional

Sofre *hoisting*, permitindo sua invocação antes da linha de declaração no código.

```
function quadrado(x) {  
  return x * x;  
}
```

2. Expressão de Função (Função Anônima)

Armazenada em uma variável. Não sofre *hoisting*.

```
const quadrado = function(x) {  
  return x * x;  
};
```

3. Arrow Functions (ES6+)

Sintaxe concisa para funções anônimas. Além da sintaxe reduzida, Arrow Functions **não** possuem seu próprio escopo `this` (herdam do contexto pai).

```
// Sintaxe completa  
const multiplicar = (a, b) => {  
  return a * b;  
};  
  
// Retorno implícito (se houver apenas uma linha)  
const somar = (a, b) => a + b;  
  
// Parênteses opcionais com apenas um parâmetro  
const dobro = x => x * 2;  
  
console.log(somar(5, 5)); // → 10  
console.log(dobro(4));   // → 8
```

Arrays e Iteração

Arrays armazenam coleções ordenadas de dados.

```
const lista = [2, 3, 5, 7, 11];  
console.log(lista[1]); // → 3
```

Abaixo, a evolução das estruturas de laços para iteração sobre arrays:

```
const frutas = ["Maçã", "Banana", "Laranja"];

// 1. Laço tradicional
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}

// 2. Laço moderno: for...of (ES6)
for (const fruta of frutas) {
  console.log(fruta);
}

// 3. Método forEach (Abordagem Funcional)
frutas.forEach(fruta => console.log(fruta));
```

Closures (Aprofundamento)

O JavaScript suporta funções de fechamento (*closures*). Uma closure é uma função que “lembra” do ambiente em que foi criada, ou seja, tem acesso às variáveis de seu escopo pai mesmo após a execução da função pai ter terminado.

```
const multiplicador = (fator) => {
  return (numero) => {
    return numero * fator;
  };
};

// 'dobro' armazena a função interna, com o 'fator' preservado como 2
const dobro = multiplicador(2);
const triplo = multiplicador(3);

console.log(dobro(5)); // → 10
console.log(triplo(5)); // → 15
```

Links Importantes e Documentação

- [Introdução ao JS - MDN](#)
- [Tipos e Estruturas de Dados](#)
- [Funções e Arrow Functions](#)
- [Trabalhando com Objetos](#)
- [Arrays e Métodos Iteradores](#)
- [Fetch API \(Consumo de APIs - Conteúdo Futuro\)](#)

Lista de Exercícios: Fundamentos de JavaScript (ES6+)

Os exercícios abaixo apresentam complexidade progressiva e devem ser resolvidos utilizando as práticas modernas da linguagem (declaração de variáveis com `let` e `const`, e utilização de *Arrow Functions* a partir do Exercício 3).

Utilize o ambiente de execução do navegador (Console) ou o Node.js para testar os scripts.

Exercício 1: Estruturas de Repetição (Triângulo)

Escreva um laço de repetição que execute exatamente **7 iterações**. A cada passagem pelo laço, o programa deve invocar o comando `console.log` **uma única vez** para exibir a linha correspondente do seguinte padrão geométrico:

```
#
##
###
####
#####
#####
#####
```

Nota Técnica: Lembre-se que você pode concatenar strings ou utilizar a propriedade `.length` na sua variável de controle para manipular a quantidade de caracteres `#` impressos em cada iteração.

Exercício 2: Laços Aninhados (Tabuleiro de Xadrez)

Escreva um programa que construa uma única string representando um quadro 8x8. Utilize o caractere de quebra de linha (`\n`) para separar as linhas. As posições do quadro devem alternar entre um espaço em branco (`" "`) e o caractere cerquilha (`"#"`), formando um padrão de tabuleiro de xadrez.

A saída no `console.log` deve ser estritamente:

```
# # # #
# # # #
# # # #
# # # #
# # # #
# # # #
# # # #
# # # #
```

Requisito Adicional: Implemente a lógica de forma que o tamanho do tabuleiro possa ser dinamicamente alterado pela modificação de uma única constante `tamanho` (ex: `const tamanho = 8;`).

Exercício 3: Arrays e Funções Modernas (Soma)

Implemente uma *Arrow Function* chamada `somaArray` que receba um array de números e retorne a soma de todos os seus elementos.

Exemplo de uso:

```
const numeros = [1, 2, 3, 4];
console.log(somaArray(numeros)); // Saída esperada: 10
```

Desafio (Opcional): Implemente a mesma função utilizando o método funcional nativo `Array.prototype.reduce()`.

Exercício 4: Manipulação de Referências (Inversão In-Place)

Crie uma *Arrow Function* chamada `inverteArray` que inverta a ordem dos elementos de um array.

Restrições Técnicas:

1. A função deve ser *in-place*, ou seja, a inversão deve ocorrer no próprio array passado como argumento, **sem a criação de um array auxiliar**.
2. Não utilize o método nativo `Array.prototype.reverse()`.

Exemplo de uso:

```
const arrayTeste = [1, 2, 3, 4, 5];
inverteArray(arrayTeste);
console.log(arrayTeste); // Saída esperada: [5, 4, 3, 2, 1]
```

Exercício 5: Iteração em Strings (Contagem de Caracteres)

Crie uma *Arrow Function* chamada `contaLetras` que receba dois parâmetros: uma string base e um caractere alvo. A função deve retornar o número de vezes que o caractere alvo aparece na string base.

Exemplo de uso:

```
const frase = "Desenvolvimento Web";
console.log(contaLetras(frase, "e")); // Saída esperada: 3
```

Exercício 6: Algoritmo de Busca (Substring)

Crie uma *Arrow Function* chamada `procuraSubstring` que receba duas strings: um texto base e um termo de busca. A função deve procurar o termo dentro do texto e retornar o índice exato onde a primeira ocorrência do termo começa. Caso o termo não exista no texto, a função deve retornar `-1`.

Restrições Técnicas: O programa **não** pode utilizar métodos nativos de busca em strings do JavaScript, tais como `indexOf()`, `includes()`, `search()`, `match()` ou expressões regulares (RegExp). A lógica deve ser implementada iterando pelos caracteres das strings.

Exemplo de uso:

```
const texto = "Universidade Federal do Paraná";
console.log(procuraSubstring(texto, "Federal")); // Saída esperada: 13
console.log(procuraSubstring(texto, "Tecnologia")); // Saída esperada: -1
```

Exercício 7: Processamento de Dados (Agrupamento e Relatório)

Este exercício simula um cenário real de desenvolvimento *Front-end*, onde a aplicação cliente recebe uma lista de dados no formato JSON (proveniente de uma API) e precisa processá-la para exibir um painel de indicadores (*Dashboard*).

Crie uma *Arrow Function* chamada `gerarRelatorio` que receba um array de objetos representando o estoque de uma loja. Cada objeto possui a seguinte estrutura: `{ id, nome, categoria, preco, quantidade }`.

A função deve processar os dados e retornar um **novo objeto** contendo as seguintes informações agregadas:

1. `valorTotalEstoque` : O valor monetário total de todos os produtos somados (`preço  $\times$  quantidade`).
2. `produtoMaisCaro` : O nome do produto com o maior preço unitário.
3. `itensPorCategoria` : Um objeto que agrupa a quantidade total de itens (unidades físicas) disponíveis em cada categoria.

Restrições Técnicas:

- A função deve ser construída utilizando métodos iteradores modernos de array (`reduce`, `filter`, `map`, `forEach`). É proibido o uso de laços tradicionais (`for` ou `while`).

Exemplo de uso e estrutura esperada:


```
const estoque = [
  { id: 1, nome: "Processador", categoria: "Hardware", preco: 1500, quantidade: 10 },
  { id: 2, nome: "Mouse", categoria: "Periféricos", preco: 150, quantidade: 30 },
  { id: 3, nome: "Teclado", categoria: "Periféricos", preco: 250, quantidade: 20 },
  { id: 4, nome: "Placa de Vídeo", categoria: "Hardware", preco: 3200, quantidade: 5 },
  { id: 5, nome: "Monitor", categoria: "Monitores", preco: 1200, quantidade: 15 }
];

const relatorio = gerarRelatorio(estoque);
console.log(relatorio);

/* Saída Esperada:
{
  valorTotalEstoque: 58500,
  produtoMaisCaro: "Placa de Vídeo",
  itensPorCategoria: {
    "Hardware": 15,
    "Periféricos": 50,
    "Monitores": 15
  }
}
*/
```

Aula 07 - Programação Cliente (Manipulação do DOM)

Bibliografia recomendada

- [MDN Web Docs - Introdução ao DOM](#)
- [w3schools - JS HTML DOM](#)

DOM (Document Object Model)

O **DOM** (Document Object Model) é uma interface de programação padronizada pelo W3C que representa o documento HTML como uma árvore de objetos (nós). Esta API permite que a linguagem JavaScript acesse, altere, adicione ou remova elementos e estilos do documento renderizado pelo navegador em tempo de execução.

Através do DOM, a estrutura estática do HTML torna-se dinâmica. O ponto de entrada principal para esta API é o objeto global `document`.

Manipulação do DOM

A implementação nativa da API nos navegadores modernos dispensa o uso de bibliotecas de terceiros (como jQuery) para a maioria das tarefas de manipulação.

Seleção de Elementos (Busca)

Historicamente, utilizavam-se métodos específicos para ID, classes ou tags. No JavaScript moderno, o padrão da indústria é a utilização dos métodos `querySelector` e `querySelectorAll`, que aceitam a sintaxe padrão de seletores CSS, conferindo maior flexibilidade.

```
// Métodos legados (ainda válidos e performáticos, mas menos flexíveis)
const porId = document.getElementById("teste");
const porClasse = document.getElementsByClassName("intro");
const porTag = document.getElementsByTagName("p");

// O padrão moderno e mais versátil: querySelector (Retorna o primeiro elemento que
casar com o seletor)
const titulo = document.querySelector("#teste");
const primeiroParagrafoIntro = document.querySelector("p.intro");

// querySelectorAll (Retorna uma NodeList com todos os elementos que casam com o
seletor)
const todosParagrafos = document.querySelectorAll("p");
```

Alteração de Conteúdo e Propriedades

Após selecionar o elemento, suas propriedades podem ser alteradas. O uso de `var` é obsoleto; utilize `const` para referências de elementos do DOM.

```
const elemento = document.querySelector("#p1");

// 1. textContent: Altera apenas o texto. É mais rápido e seguro contra ataques XSS.
elemento.textContent = "Novo conteúdo textual puro!";

// 2. innerHTML: Interpreta tags HTML. Use apenas se for estritamente necessário
renderizar novas tags.
elemento.innerHTML = "Texto com <strong>negrito</strong>.";

// 3. Alteração de atributos
const imagem = document.querySelector("#myImage");
imagem.src = "landscape.jpg";
imagem.alt = "Nova descrição da imagem";

// 4. Alteração de estilos inline (Apenas para valores dinâmicos)
const p2 = document.querySelector("#p2");
p2.style.color = "blue";
p2.style.marginTop = "20px";

// 5. Manipulação de Classes CSS (Prática recomendada para estilização)
const divAlerta = document.querySelector(".alerta");
divAlerta.classList.add("alerta-sucesso"); // Adiciona classe
divAlerta.classList.remove("alerta-erro"); // Remove classe
divAlerta.classList.toggle("oculto");      // Alterna a classe (adiciona se não tiver,
remove se tiver)
```

Nota: Evite o uso da função `document.write()`. Trata-se de uma prática obsoleta que sobrescreve todo o documento HTML caso executada após o carregamento da página.

Navegação no DOM (Traversing)

É possível navegar pela árvore do DOM através de relações estruturais (pais, filhos e irmãos). Recomenda-se utilizar as propriedades baseadas em *Element* em vez de *Node*, pois ignoram nós de texto em branco (quebras de linha no HTML).

```
const container = document.querySelector("#demo");

// Navegação Moderna (Focada em Elementos HTML)
const pai = container.parentElement;
const filhos = container.children; // Retorna uma HTMLCollection apenas com tags
filhas
const primeiroFilho = container.firstChild;
const ultimoFilho = container.lastElementChild;
const proximoIrmão = container.nextElementSibling;
const irmaoAnterior = container.previousElementSibling;

// Acesso direto ao primeiro filho
const textoPrimeiroFilho = filhos[0].textContent;
```

Criação de Elementos Dinâmicos

A inserção de novos componentes na interface envolve a criação de elementos na memória e sua posterior anexação à árvore do DOM.

```
// 1. Criação de um novo elemento <div>
const novaDiv = document.createElement("div");

// 2. Configuração do elemento
novaDiv.textContent = "Conteúdo gerado via JS";
novaDiv.classList.add("box-destaque");

// 3. Seleção do nó pai onde o elemento será inserido
const elementoPai = document.querySelector("#secao-principal");

// 4. Inserção no DOM
elementoPai.append(novaDiv); // append() é o padrão moderno, substituindo
appendChild()
```

Eventos e Interatividade

A interatividade no lado do cliente ocorre por meio do mapeamento de **eventos** (cliques, digitação, envio de formulários, redimensionamento de janela).

Má Prática (Evite): O uso de atributos de evento diretamente no HTML (ex: `<button onclick="funcao()">`) fere o princípio de separação de responsabilidades. O HTML deve conter apenas a estrutura, e o JS o comportamento.

Padrão Moderno (`addEventListener`):

A interface `addEventListener` permite vincular múltiplas funções a um mesmo evento sem sobrescrever ouvintes anteriores.

```
// Seleção do botão
const botaoEnviar = document.querySelector("#btnEnviar");

// Declaração da função de callback (Arrow Function)
const processarClique = (event) => {
  // O objeto 'event' (ou 'e') contém os metadados do evento disparado

  // event.target aponta para o elemento exato que acionou o evento
  event.target.textContent = "Processando...";
  event.target.classList.add("desativado");

  console.log("Botão clicado nas coordenadas: X:", event.clientX, "Y:",
    event.clientY);
};

// Vinculação do ouvinte de evento
botaoEnviar.addEventListener("click", processarClique);
```

Atribuindo eventos a múltiplos elementos

Se o sistema precisar capturar eventos de uma lista de elementos semelhantes, utilize um laço de repetição (`forEach`) sobre a `NodeList` gerada pelo `querySelectorAll`.

```
// Seleciona todos os parágrafos com a classe 'clicavel'
const paragrafos = document.querySelectorAll("p.clicavel");

paragrafos.forEach((paragrafo) => {
  paragrafo.addEventListener("click", (e) => {
    // Altera a cor de fundo do elemento específico que foi clicado
    e.target.style.backgroundColor = "yellow";
  });
});
```

Links Importantes

- [Introdução a Eventos](#)
- [Manipulando o DOM](#)
- [Element.classList](#)
- [Document.querySelector](#)

Exercícios Práticos: JavaScript, DOM e Eventos

Nível 1: Fundamentos da Interatividade

Exercício 1: Alerta de Boas-Vindas Personalizado

- **Objetivo:** Introduzir a captura de valor de um `input` e a resposta a um clique de botão.
 - **Descrição:** Crie uma página HTML com:
 1. Um campo de texto (`<input type="text">`) com um `id` para que o usuário possa digitar seu nome.
 2. Um botão (`<button>`) com o texto “Enviar”.
 3. Ao clicar no botão, seu script JavaScript deve ler o nome digitado no campo de texto e exibir um alerta de boas-vindas na tela, como por exemplo: Olá, [Nome do Usuário]! Bem-vindo(a)! .
 - **Conceitos-chave:** `document.getElementById()` , `addEventListener('click', ...)` , `.value` , `alert()` .
 - **Desafio Bônus:** Se o usuário clicar no botão sem ter digitado nada, exiba um alerta diferente pedindo para que ele preencha o nome.
-

Exercício 2: Paleta de Cores

- **Objetivo:** Praticar a manipulação de estilos CSS através do JavaScript.
 - **Descrição:** Desenvolva uma página com:
 1. Três ou mais botões, cada um representando uma cor (ex: “Vermelho”, “Verde”, “Azul”).
 2. Ao clicar em um dos botões, o `<body>` da página deve ter sua cor de fundo (`backgroundColor`) alterada para a cor correspondente.
 - **Conceitos-chave:** `document.querySelector()` , `element.style` , manipulação de múltiplos elementos.
 - **Desafio Bônus:** Adicione um botão “Aleatório” que, ao ser clicado, muda a cor de fundo para uma cor hexadecimal gerada aleatoriamente (ex: `#f1f5f8`).
-

Exercício 3: Contador de Cliques

- **Objetivo:** Manipular o conteúdo de texto de um elemento e trabalhar com variáveis em JavaScript.
- **Descrição:** Crie uma página com:
 1. Um parágrafo (`<p>`) que inicia exibindo o número `0` .
 2. Um botão com o texto “Incrementar”.
 3. Cada vez que o botão for clicado, o número exibido no parágrafo deve ser incrementado em 1.
- **Conceitos-chave:** `textContent` ou `innerText` , declaração e atualização de variáveis.

- **Desafio Bônus:** Adicione um segundo botão, “Decrementar”, que diminui o valor do contador. O contador nunca deve ficar abaixo de zero.
-

Nível 2: Manipulação Dinâmica e Formulários

Exercício 4: Lista de Tarefas (To-Do List)

- **Objetivo:** Praticar a criação, adição e remoção de elementos no DOM de forma dinâmica.
 - **Descrição:** Construa uma interface de lista de tarefas que permita ao usuário:
 1. Digitar uma nova tarefa em um campo de texto (`<input type="text">`).
 2. Clicar em um botão “Adicionar” para que a tarefa seja inserida como um novo item (`<li>`) em uma lista não ordenada (`<ul>`) na página.
 3. Cada item da lista deve ter um botão “Remover” ao seu lado. Ao ser clicado, este botão deve excluir o item da lista correspondente.
 - **Conceitos-chave:** `document.createElement()`, `element.appendChild()`, `element.remove()`, `event.target`.
 - **Desafio Bônus:** Adicione uma funcionalidade para que, ao clicar no texto de uma tarefa, ela seja marcada como “concluída” (por exemplo, aplicando um estilo de texto riscado com `text-decoration: line-through`).
-

Exercício 5: Validação de Formulário em Tempo Real

- **Objetivo:** Utilizar eventos de formulário e teclado para fornecer feedback instantâneo ao usuário.
 - **Descrição:** Crie um formulário de cadastro com campos para **nome**, **e-mail** e **senha**. Implemente as seguintes validações que acontecem enquanto o usuário digita:
 1. **Nome:** Não pode estar vazio.
 2. **E-mail:** Deve conter o caractere `@`.
 3. **Senha:** Deve ter no mínimo 8 caracteres. Abaixo de cada campo, exiba uma mensagem de erro (ex: em um `<p>` com cor vermelha) assim que a regra for violada. A mensagem deve desaparecer quando o campo se tornar válido novamente.
 - **Conceitos-chave:** `addEventListener('keyup', ...)` OU `addEventListener('input', ...)`, `element.classList` para estilização, condicionais, `.length`, `.includes()`.
 - **Desafio Bônus:** Adicione um campo de “Confirmar Senha” e valide se o seu valor é idêntico ao do campo “Senha”. Além disso, desabilite o botão de “Cadastrar” do formulário enquanto houver algum erro de validação.
-

Exercício 6: Galeria de Imagens com Modal

- **Objetivo:** Criar uma experiência de usuário mais rica com a exibição de um modal.
- **Descrição:** Desenvolva uma galeria com várias imagens em miniatura.
 1. Crie uma seção na página com pelo menos 4 imagens pequenas (`<img>`).

2. Ao clicar em qualquer uma das miniaturas, uma janela modal deve aparecer, sobrepondo o restante do conteúdo.
 3. O modal deve conter a imagem que foi clicada, porém em um tamanho maior.
 4. O modal também deve ter um botão "Fechar" (um "X" no canto, por exemplo) que, ao ser clicado, o esconde.
- **Conceitos-chave:** Manipulação da propriedade `display` ou de classes para mostrar/esconder elementos, capturar atributos (`getAttribute('src')`), estrutura HTML/CSS para overlays.
 - **Desafio Bônus:** Permita que o usuário feche o modal também ao pressionar a tecla `Esc` no teclado ou ao clicar fora da área da imagem (no overlay).
-

Nível 3: Aplicações Complexas e Gerenciamento de Estado

Exercício 7: Jogo da Velha (Tic-Tac-Toe)

- **Objetivo:** Gerenciar o estado de um jogo, implementar lógica de vitória e reiniciar o estado da aplicação.
 - **Descrição:** Crie um jogo da velha funcional. A interface deve ser um grid 3x3 (pode ser feito com `<div>`s ou `<button>`s).
 1. Os jogadores, "X" e "O", devem se alternar a cada clique em uma célula vazia.
 2. Após cada jogada, o script deve verificar se houve um vencedor (três marcas iguais em qualquer linha, coluna ou diagonal).
 3. Se houver um vencedor, exiba uma mensagem (ex: "O jogador X venceu!") e impeça novas jogadas.
 4. Se todas as células forem preenchidas sem um vencedor, declare um empate.
 5. Adicione um botão "Reiniciar Jogo" que limpa o tabuleiro e reinicia a lógica.
 - **Conceitos-chave:** Arrays para representar o estado do tabuleiro, lógica condicional complexa, gerenciamento de turnos, desabilitar elementos (`element.disabled`).
 - **Desafio Bônus:** Mantenha um placar que mostre quantas vitórias cada jogador (X e O) acumulou durante a sessão.
-

Exercício 8: Arrastar e Soltar (Drag and Drop)

- **Objetivo:** Implementar interações avançadas utilizando a API de Drag and Drop do HTML5.
- **Descrição:** Crie uma interface com duas colunas principais.
 1. A primeira coluna, "A Fazer", deve conter uma lista de itens (ex: `<div>`s com texto). Esses itens devem ser "arrastáveis".
 2. A segunda coluna, "Concluído", deve ser uma "área de soltura".
 3. O usuário deve ser capaz de clicar e arrastar um item da coluna "A Fazer" e soltá-lo dentro da coluna "Concluído". Ao ser solto, o item deve se mover permanentemente para a nova coluna.
- **Conceitos-chave:** Atributo `draggable="true"`, eventos `dragstart`, `dragover`, `drop`, `event.preventDefault()`, `event.dataTransfer`.
- **Desafio Bônus:** Permita que os itens também possam ser arrastados de volta da coluna "Concluído" para a coluna "A Fazer". Adicione um feedback visual (ex: uma borda pontilhada) na coluna "Concluído" quando um item estiver sendo arrastado sobre ela.

Exercício 9: Filtro Dinâmico de Produtos

- **Objetivo:** Simular uma aplicação de e-commerce, manipulando um conjunto de dados e atualizando a UI dinamicamente.
- **Descrição:** Crie uma página para exibir uma lista de produtos.
 1. No seu arquivo JavaScript, crie um array de objetos. Cada objeto representará um produto e deve ter propriedades como `id`, `nome`, `preco` e `categoria` (ex: "Eletrônicos", "Roupas").
 2. Renderize todos os produtos na página, criando os elementos HTML correspondentes.
 3. Adicione os seguintes controles de filtro na página:
 - Um campo de busca (`<input type="text">`) para filtrar por nome.
 - Um seletor (`<select>`) ou checkboxes para filtrar por categoria.
 - Um `input` do tipo `range` para filtrar por preço máximo.
 4. À medida que o usuário interage com **qualquer um** dos filtros, a lista de produtos exibida na tela deve ser atualizada em tempo real para mostrar apenas os itens que correspondem a **todos** os critérios selecionados.
- **Conceitos-chave:** Manipulação de arrays (`filter()`, `map()`, `forEach()`), combinação de múltiplos eventos (`input`, `change`), renderização dinâmica a partir de um array de dados.
- **Desafio Bônus:** Adicione botões para ordenar os produtos filtrados por preço (menor para maior e maior para menor).

Aula 8 - JQuery

Bibliografia recomendada para o tema:

- [w3schools - JQuery](#);
- [Jquery oficial](#).

Repositório com *starter code* para a aula:

<https://gitlab.com/ds122-alexkutzke/ds122-jquery-example>

Jquery

Jquery é uma biblioteca Javascript que procura facilitar tarefas recorrentes utilizadas em programas dessa linguagem. A intenção dessa biblioteca é permitir ao programador “escrever menos e fazer mais”. Dentre as funcionalidades da linguagem estão:

- Manipulação HTML/DOM;
- Manipulação CSS;
- Eventos HTML;
- Efeitos e animações;
- AJAX;
- Utilidades.

Para utilizar o JQuery, é necessário adicionar o arquivo JS da biblioteca no HTML. Isso pode ser feito de duas formas diferentes:

```
<!-- Baixar o arquivo da biblioteca e adicionar no HTML -->
<head>
<script src="jquery-3.1.1.min.js"></script>
</head>

<!-- Utilizando um CDN (Content delivery network) -->
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.1.1/jquery.min.js">
</script>
</head>
```

Sintaxe

A sintaxe básica da biblioteca JQuery é a seguinte: `$(selector).action()` , onde:

- O sinal `$` define o acesso ao jQuery (pode, inclusive, ser substituído por `JQuery`);
- O seletor é uma string para encontrar elementos HTML no DOM;
- A ação é algo para ser realizado sobre os elementos encontrados com o seletor.

```
$(this).hide() // esconde o elemento "atual".

$("p").hide() // esconde todos os parágrafos.

$(".test").hide() // esconde todos os elementos com class="test".

$("#test").hide() // esconde todos os elementos com id="test".
```

Seletores

Dentre os seletores mais utilizados, estão os seletores iguais ao do CSS. Como `.` para classes, `#` para ids, etc.

Para mais seletores, consulte: https://www.w3schools.com/jquery/jquery_selectors.asp

Eventos

Jquery facilita o tratamento de eventos com Javascript. Um dos mais utilizados é o `ready`

```
$(document).ready(function(){

    // jQuery methods go here...

});
```

No código acima, o conteúdo da função será executado apenas quando a página estiver “pronta”, ou seja, quando todos os elementos do documento já foram carregados. O mesmo código acima pode ser escrito com o seguinte “atalho”:

```
$(function(){

    // jQuery methods go here...

});
```

Além do evento `ready` o Jquery permite a manipulação de qualquer outro evento:

```
// Esconde um parágrafo quando este for clicado.
$("p").click(function(){
    $(this).hide();
});

// Exibe duas mensagens: uma quando o mouse passa pelo elemento #p1 e outra quando o
mouse sai do elemento
$("#p1").hover(function(){
    alert("You entered p1!");
},
function(){
    alert("Bye! You now leave p1!");
});
```

Outra forma de atribuir eventos a elementos com Jquery é por meio do método `on` :

```

$("p").on("click", function(){
    $(this).hide();
});

// Atribui mais de um evento em uma única chamada ao método "on"
$("p").on({
    mouseenter: function(){
        $(this).css("background-color", "lightgray");
    },
    mouseleave: function(){
        $(this).css("background-color", "lightblue");
    },
    click: function(){
        $(this).css("background-color", "yellow");
    }
});

```

O método `on` é preferível. Ele cria apenas uma única referência para a função do evento para todos os elementos selecionados. Isso permite uma economia de memória no cliente. Além disso, elementos criados dinamicamente (após a página ter sido carregada pelo navegador), respondem a eventos criados com o `on`. O mesmo não acontece com métodos como o `click`. Nesse caso, para cada novo elemento criado, uma nova chamada para `click` teria que ser realizada. Mais sobre eventos sobre elementos criados dinamicamente: <http://api.jquery.com/on/#direct-and-delegated-events>

Para mais eventos, consulte: https://www.w3schools.com/jquery/jquery_ref_events.asp

Manipulação do DOM com JQuery

Exemplos de manipulação do DOM com JQuery:

```
// Esconder e mostrar
$("#hide").click(function(){
    $("#p").hide();
});

$("#show").click(function(){
    $("#p").show();
});

//-----
$("#btn2").click(function(){
    alert("HTML: " + $("#test").html());

    $(this).html("Novo conteúdo");
});

//-----
$("#btn1").click(function(){
    alert("Value: " + $("#test").val());
});

// -----
function appendText() {
    var txt1 = "<p>Text.</p>"; // Create element with HTML
    var txt2 = $("<p></p>").text("Text."); // Create with jQuery
    var txt3 = document.createElement("p"); // Create with DOM
    txt3.innerHTML = "Text.";
    $("body").append(txt1, txt2, txt3); // Append the new elements
}

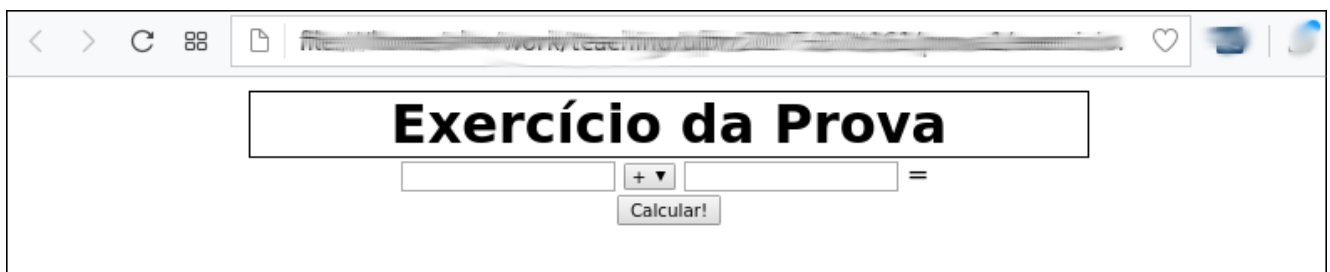
//-----
$("#p").css("background-color", "yellow");
```

Links interessantes

- [Documentação do JQuery.](#)

Exercícios de provas anteriores

1. Considere a seguinte página web:



The screenshot shows a web browser window with a single page titled "Exercício da Prova". The page contains a simple calculator interface. At the top, there is a title "Exercício da Prova" in a large, bold, black font. Below the title, there is a form with two input fields for numbers, a dropdown menu for selecting a mathematical operation (currently showing a plus sign), and an equals sign. Below the form, there is a button labeled "Calcular!". The browser's address bar shows a URL that appears to be "http://localhost:3000/work/teaching/001/2017-2018/".

O funcionamento dela é simples. O usuário pode inserir um número inteiro em cada um dos campos de texto e selecionar uma das 4 operações matemáticas básicas (soma, subtração, multiplicação e divisão) no seletor ao centro. Em seguida, ao clicar em "Calcular!" a página resolve a expressão matemática definida pelo usuário e exibe o resultado na caixa abaixo. Nesse contexto, implemente a página web descrita em 3 arquivos diferentes, sendo eles:

- Um arquivo HTML com a estrutura da página;

- Um arquivo CSS com as definições de estilo da página, para que ela se pareça com a imagem acima;
- Um arquivo JS com a programação necessária para que o comportamento da página seja o mesmo exposto no enunciado acima.

Os três arquivos devem estar relacionados entre si. Ou seja, o arquivo HTML deve fazer referência aos outros dois.

Para ajudar, segue a sintaxe de algumas tags HTML e uma função JS:

```
# A funcao parseInt() transforma uma string em um inteiro:  
  
var x = parseInt("123");  
# x -> 123
```

2. Considere a seguinte página web:

Exercício da Prova

A frase contem:
9 caracteres;
4 vogais;
4 consoantes
1 espaço.

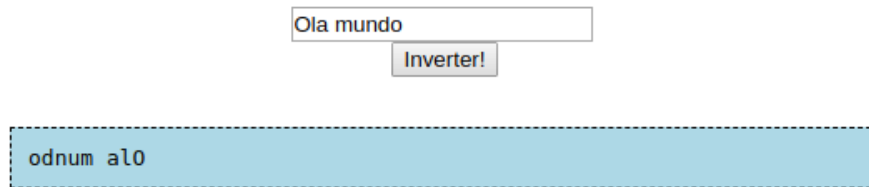
O funcionamento dela é simples. O usuário pode inserir uma frase na caixa de texto (considere que a frase contém apenas letras e espaços). Em seguida, ao clicar em “Contar!” a página exibe a contagem total de caracteres, vogais, consoantes e espaços contidos na frase inserida pelo usuário. Nesse contexto, implemente a página web descrita em 3 arquivos diferentes, sendo eles:

- Um arquivo HTML com a estrutura da página;
- Um arquivo CSS com as definições de estilo da página, para que ela se pareça com a imagem acima;
- Um arquivo JS com a programação necessária para que o comportamento da página seja o mesmo exposto no enunciado acima.

Os três arquivos devem estar relacionados entre si. Ou seja, o arquivo HTML deve fazer referência aos outros dois.

3. Considere a seguinte página web:

Exercício da Prova



The screenshot shows a web page with a title 'Exercício da Prova' and a horizontal line below it. Below the line is a text input field containing 'Ola mundo' and a button labeled 'Inverter!'. Below the button is a dashed rectangular box containing the text 'odnum a10'.

O funcionamento dela é simples. O usuário pode inserir uma frase na caixa de texto (considere que a frase contém apenas letras e espaços). Em seguida, ao clicar em “Inverter!” a página exibe a frase inserida pelo usuário ao contrário. Nesse contexto, implemente a página web descrita em 3 arquivos diferentes, sendo eles:

- Um arquivo HTML com a estrutura da página;
- Um arquivo CSS com as definições de estilo da página, para que ela se pareça com a imagem acima;
- Um arquivo JS com a programação necessária para que o comportamento da página seja o mesmo exposto no enunciado acima.

Os três arquivos devem estar relacionados entre si. Ou seja, o arquivo HTML deve fazer referência aos outros dois.

Aula 09 - Frameworks Front-end

Bibliografia recomendada para o tema:

Frameworks

- [Bootstrap \(versão 3\)](#): famoso framework front-end;
 - [Guia bootstrap do w3schools](#);
- [Materialize](#): baseado no Material Design;
- [W3.CSS](#): oficial do w3schools

Outros recursos

- <http://shoelace.io>

Starter-code

<http://gitlab.com/ds122-alexkutzke/ds122-bootstrap-example>

Framworks Front-end

Front-end é o termo utilizado para se fazer referência à parte de uma aplicação Web que é executadas na máquina cliente. Ou seja, em geral, envolvem as tecnologias HTML, CSS e Javascript e se tratam, em grande medida, de detalhes da **interface** dos sistemas Web.

Frameworks front-end são, por sua vez, conjuntos de especificações, arquivos e bibliotecas utilizados para facilitar o desenvolvimento de front-end's para aplicações Web. Por exemplo, um framework front-end pode ser um conjunto de arquivos CSS e JS que podem ser utilizados em diferentes aplicações Web, desde de que algumas regras para sua utilização sejam seguidas.

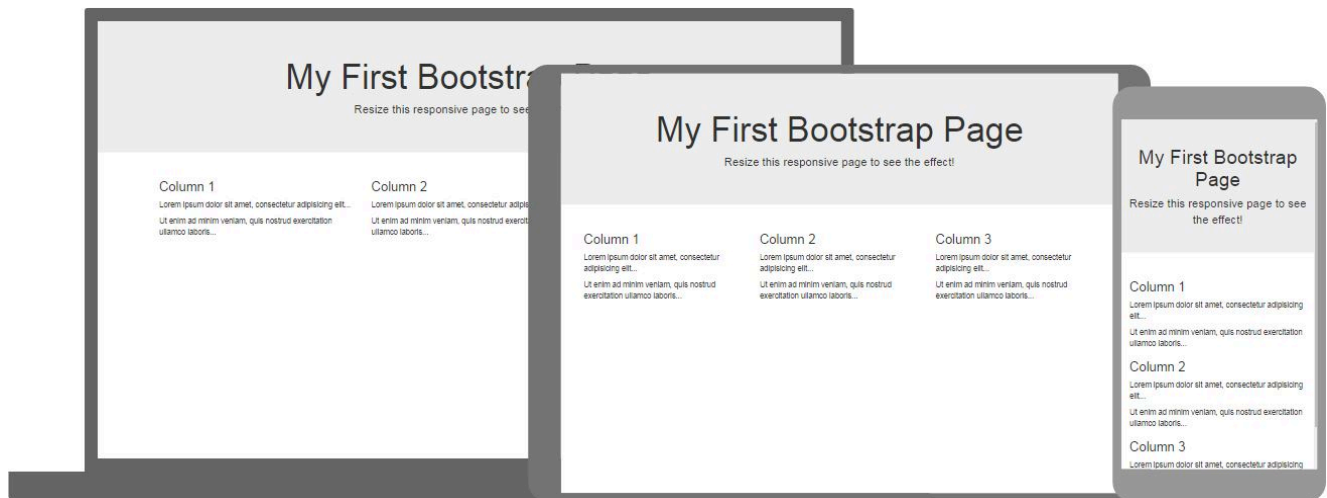
Em geral, o uso de um framework front-end envolve os seguintes passos:

- Carregamento dos arquivos CSS e JS do framework (por CDN ou download);
- Organização dos arquivos HTML de acordo com as especificações do framework;
- Utilização de classes CSS e funções JS disponibilizadas pelo framework.

Vários frameworks front-end são disponibilizados na internet. O mais famoso deles é conhecido por **Bootstrap**.

Bootstrap

Bootstrap é o mais famoso framework front-end para desenvolvimento Web. Com ele é possível criar aplicações **responsivas** e **mobile-first**.



Suas principais características são:

- Facilidade de uso;
- Responsividade;
- Abordagem "mobile-first";
- Compatibilidade com a maioria dos navegadores modernos.

Para utilizar o Bootstrap, é possível carregar seus arquivos a partir de CDN:

```
<!-- Latest compiled and minified CSS -->
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">

<!-- jQuery library -->
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.0/jquery.min.js">
</script>

<!-- Latest compiled JavaScript -->
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js">
</script>
```

A partir daí, basta utilizar as classes CSS e as funções JS do Bootstrap nos seus documentos HTML. Por exemplo:

```

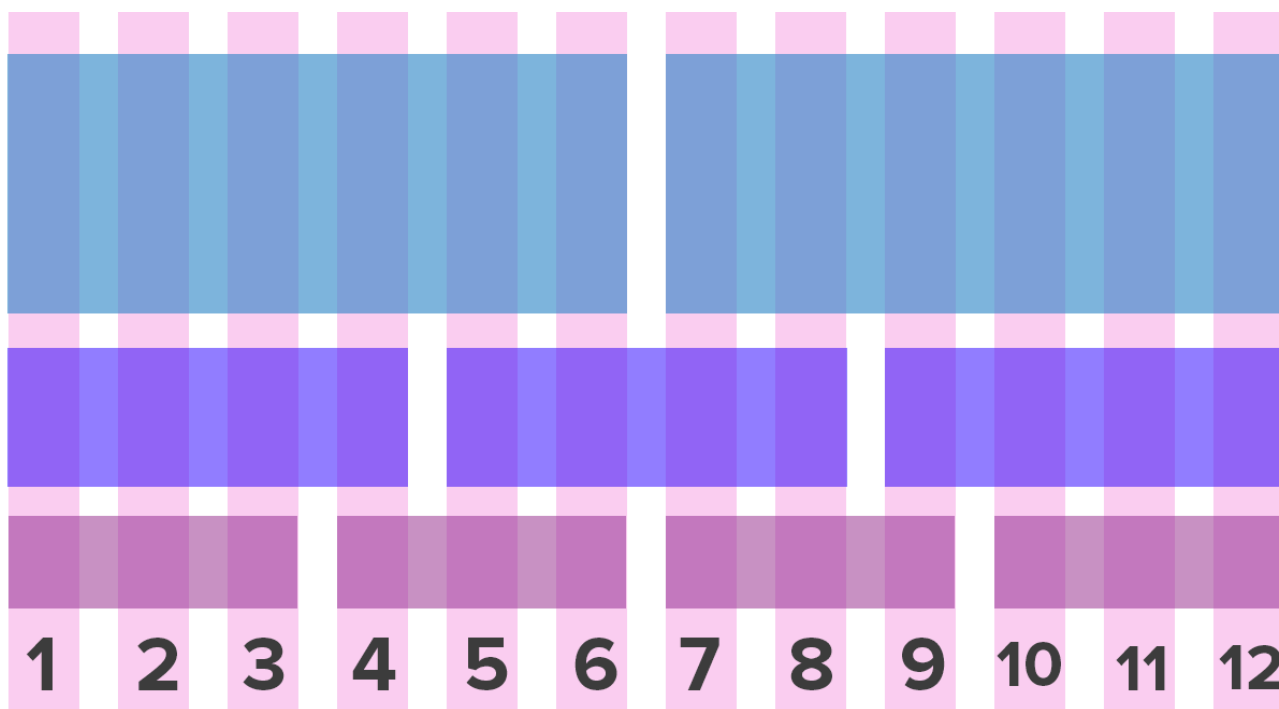
<div class="jumbotron text-center">
  <h1>My First Bootstrap Page</h1>
  <p>Resize this responsive page to see the effect!</p>
</div>

<div class="container">
  <div class="row">
    <div class="col-sm-4">
      <h3>Column 1</h3>
      <p>Lorem ipsum dolor..</p>
      <p>Ut enim ad..</p>
    </div>
    <div class="col-sm-4">
      <h3>Column 2</h3>
      <p>Lorem ipsum dolor..</p>
      <p>Ut enim ad..</p>
    </div>
    <div class="col-sm-4">
      <h3>Column 3</h3>
      <p>Lorem ipsum dolor..</p>
      <p>Ut enim ad..</p>
    </div>
  </div>
</div>

```

Sistema de grid

Um dos conceitos chave para a disposição de elementos utilizando o bootstrap é sistema de grid. Este sistema considera que toda interface criada com o Bootstrap será dividida virtualmente em 12 colunas de mesma largura. Além disso, qualquer subconjunto dessas colunas podem ser divididas, novamente, em 12 partes iguais.



Esse sistema de grid permite a criação facilitada de colunas em uma interface web. Outro ponto importante é que tal sistema garante que as colunas sejam responsivas, ou seja, se comportem da melhor maneira possível de acordo com o tamanho de tela do dispositivo que acessa o sistema.

Para se utilizar o sistema de grids, basta utilizar as classes do tipo `col-TIPO-LARGURA`, onde:

- TIPO:
 - xs : para smartphones;
 - sm : para tablets;
 - md : para desktops;
 - lg : para tv's e afins;
- LARGURA
 - Um tamanho de 1 a 12;

A especificação do bootstrap (<http://getbootstrap.com/css/#grid>) relata que as páginas que utilizam o framework devem ser compostas por blocos (containers) que são divididos em partes horizontais (rows) e que cada uma dessas partes podem ser compostas por diferentes disposições de colunas. A seguir, alguns exemplos de colunas com bootstrap.

```
<div class="container">
  <div class="row">
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
    <div class="col-md-1">.col-md-1</div>
  </div>
  <div class="row">
    <div class="col-md-8">.col-md-8</div>
    <div class="col-md-4">.col-md-4</div>
  </div>
  <div class="row">
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
  </div>
  <div class="row">
    <div class="col-md-6">.col-md-6</div>
    <div class="col-md-6">.col-md-6</div>
  </div>
</div>
```

.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1
.col-md-8								.col-md-4			
.col-md-4				.col-md-4				.col-md-4			
.col-md-6						.col-md-6					

Exemplo de colunas para mais de um tamanho de dispositivo:

```
<!-- Stack the columns on mobile by making one full-width and the other half-width -->
<div class="row">
  <div class="col-xs-12 col-md-8">.col-xs-12 .col-md-8</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
</div>

<!-- Columns start at 50% wide on mobile and bump up to 33.3% wide on desktop -->
<div class="row">
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
  <div class="col-xs-6 col-md-4">.col-xs-6 .col-md-4</div>
</div>

<!-- Columns are always 50% wide, on mobile and desktop -->
<div class="row">
  <div class="col-xs-6">.col-xs-6</div>
  <div class="col-xs-6">.col-xs-6</div>
</div>
```

Para saber mais sobre funcionalidades do Bootstrap, consulte o site oficial do framework. Existem classes para praticamente qualquer tag HTML.

Aula 10 - Programação Back-end

Programação Back-end

Back-end, ou *server-side*, é o termo utilizado para fazer referência à parte de uma aplicação Web que é executada na máquina servidor. Em sua essência, envolve as operações relacionadas ao processamento da lógica de negócio e à produção e entrega de arquivos HTML com conteúdo dinâmico. Trata, em grande medida, do gerenciamento e da persistência dos **dados** da aplicação Web. Por essa razão, o back-end atua quase sempre integrado a sistemas de bancos de dados. Para que uma aplicação Web seja capaz de gerar conteúdo dinâmico baseando-se na arquitetura de renderização no servidor, é necessário o seguinte conjunto:

- Um servidor Web que receba e responda a requisições HTTP/HTTPS;
- Um interpretador ou programa capaz de processar as regras de negócio e produzir o código HTML dinâmico a partir da requisição do cliente;
- Um serviço de banco de dados para armazenar e fornecer os dados requisitados pelo programa.

Servidores Web

Um **servidor web** é um software que “escuta” constantemente por requisições de rede e as responde à medida que são recebidas. O padrão é que esses servidores escutem requisições HTTP na porta 80 e requisições seguras HTTPS na porta 443.

Portanto, a função básica de um servidor web é: dada uma requisição do cliente, localizar ou processar o recurso solicitado e enviá-lo de volta embutido em uma resposta HTTP.

Alguns servidores Web amplamente utilizados são:

- Apache HTTP Server (<https://httpd.apache.org/>);
- Nginx (<https://www.nginx.com/>);
- Apache Tomcat (<http://tomcat.apache.org/>);
- Lighttpd (<http://www.lighttpd.net/>).

Tradução de URL para arquivos locais ao servidor

No modelo tradicional de entrega de páginas, ao receber uma requisição, o servidor web procura em seus arquivos locais o recurso a ser enviado ao cliente. Por exemplo, ao receber uma requisição GET para a URL `http://example.com/dir1/dir2/arquivoX.html`, o servidor tentará localizar esse arquivo na estrutura de pastas contida dentro de seu **diretório raiz** (*Document Root*).

O **diretório raiz** é a pasta configurada na máquina do servidor onde ficam armazenados os arquivos públicos do site. Em servidores Apache rodando no Linux, o diretório raiz padrão geralmente é o `/var/www/html`. Dessa forma, para a requisição GET citada, o servidor entregaria o arquivo localizado no caminho físico: `/var/www/html/dir1/dir2/arquivoX.html`.

Pontos importantes sobre este processo:

- Por questões de segurança, o servidor web é configurado para negar acesso a qualquer arquivo ou diretório que esteja fora da hierarquia do seu diretório raiz.
- Os recursos entregues não se limitam a arquivos HTML. O servidor web pode entregar imagens, arquivos PDF, folhas de estilo (CSS) e scripts de front-end, dependendo da requisição.

Páginas dinâmicas

O processo descrito acima refere-se à entrega de recursos estáticos, onde o servidor apenas lê um arquivo do disco e o envia exatamente como foi salvo.

No entanto, aplicações web dependem de conteúdo dinâmico. Em um sistema de notas, por exemplo, o HTML retornado deve conter as informações específicas do usuário logado. Como seria inviável criar um arquivo HTML estático para cada usuário, o servidor precisa gerar esse conteúdo sob demanda.

Para a produção dinâmica, o servidor web trabalha em conjunto com módulos ou interpretadores de linguagem. **Não** é o servidor web (como o Apache) que processa a lógica de negócio ou acessa o banco de dados. O fluxo ocorre da seguinte forma: o servidor web recebe a requisição, identifica que o arquivo solicitado contém código dinâmico e repassa esse arquivo para o interpretador da linguagem.

Historicamente, essa comunicação entre o servidor web e os programas locais era feita através de um padrão chamado **CGI (Common Gateway Interface)**. Esse método era ineficiente, pois abria um novo processo no sistema operacional para cada acesso. Atualmente, os servidores web utilizam módulos integrados (como o `mod_php` no Apache) ou serviços de processamento dedicados (como o FastCGI/PHP-FPM) para executar as linguagens de programação com alto desempenho, gerando o HTML final e devolvendo-o ao servidor web, que o repassa ao cliente.

PHP

O PHP (PHP: Hypertext Preprocessor) é uma linguagem de script desenhada especificamente para o desenvolvimento Web e que atua do lado do servidor. Sendo de código aberto e amplamente suportada, é uma das linguagens mais tradicionais para a geração de arquivos HTML dinâmicos.

Ao receber uma requisição por um arquivo `.php`, o servidor web aciona o interpretador do PHP. Este interpretador lê o arquivo de cima a baixo, executando exclusivamente os blocos de código delimitados pelas marcações da linguagem. O que estiver fora dessas marcações é tratado como texto puro (geralmente HTML). Ao final do processamento, o PHP devolve um arquivo HTML estático completo para o servidor enviar ao navegador do usuário.

Exemplo de um arquivo `index.php`:

```
<!DOCTYPE html>
<html>
<body>

<?php
$message = "Meu primeiro script PHP!";
echo "<p>" . $message . "</p>";
?>

</body>
</html>
```

Ao passar o arquivo acima pelo interpretador, a lógica é processada e o cliente (navegador) recebe apenas o seguinte código final:

```
<!DOCTYPE html>
<html>
<body>

<p>Meu primeiro script PHP!</p>

</body>
</html>
```

Preparação do ambiente de desenvolvimento no Linux

Para desenvolver localmente, é necessário instalar um servidor web e o interpretador da linguagem. Em distribuições Linux, a abordagem mais comum é a instalação da pilha **LAMP** (Linux, Apache, MariaDB/MySQL e PHP).

Exibição de erros no PHP

Em ambientes de produção, os erros de código ficam ocultos do usuário final por segurança. Durante o desenvolvimento, é fundamental que o PHP exiba esses erros no navegador para facilitar a correção.

Para ativar a exibição, localize o arquivo de configuração `php.ini`. Em sistemas modernos utilizando Apache, o caminho costuma ser algo como `/etc/php/8.x/apache2/php.ini` (onde `8.x` corresponde à versão instalada).

Abra o arquivo, localize a opção `display_errors` e altere o seu valor para `On`:

```
display_errors = On
```

Após salvar o arquivo, é necessário reiniciar o serviço do Apache para que as novas configurações entrem em vigor. Em distribuições Linux atuais, o comando é:

```
sudo systemctl restart apache2
```

Preparação do ambiente de desenvolvimento no Windows

No Windows, a forma mais prática de configurar o ambiente de desenvolvimento é através da instalação de *bundles* (pacotes All-in-One), que instalam e pré-configuram o servidor Web, o banco de dados e o PHP simultaneamente.

Os pacotes mais comuns utilizados para este fim são:

- [XAMPP](#) - Apache + MariaDB + PHP + Perl
- [Laragon](#) - Alternativa moderna e otimizada ao XAMPP/WAMP para Windows.
- [WAMP](#) - Apache + PHP + MySQL

Modelos atuais: APIs e Arquiteturas Modernas

Embora o modelo de geração de páginas HTML no servidor (conhecido como *Server-Side Rendering* ou SSR) seja o pilar fundamental para o entendimento da arquitetura Web e ainda amplamente utilizado, o desenvolvimento de software moderno adotou também arquiteturas mais descentralizadas.

Para fins de contexto histórico, é importante saber que em disciplinas futuras você estudará um padrão arquitetural diferente, focado em **APIs** e **Single Page Applications (SPAs)**. Nesse cenário, a responsabilidade de gerar o visual da página muda de lugar:

- **API (*Application Programming Interface*):** O Back-end deixa de misturar lógica com HTML. Sua única função passa a ser processar regras de negócio, consultar o banco de dados e devolver apenas os **dados puros**. Esses dados transitam pela rede em um formato de texto universal e leve chamado **JSON** (*JavaScript Object Notation*).
- **SPA (*Single Page Application*):** O Front-end torna-se uma aplicação JavaScript robusta e independente (utilizando tecnologias como React, Angular ou Vue). Ele roda diretamente no navegador do usuário, pede os dados (JSON) para a API e “desenha” as telas dinamicamente, sem a necessidade de recarregar a página a cada clique.

Aula 11 - PHP

Bibliografia recomendada para o tema:

- [Documentação Oficial do PHP \(pt-BR\)](#)
- [PHP The Right Way \(Boas Práticas Modernas\)](#)
- [W3Schools - PHP Tutorial](#)

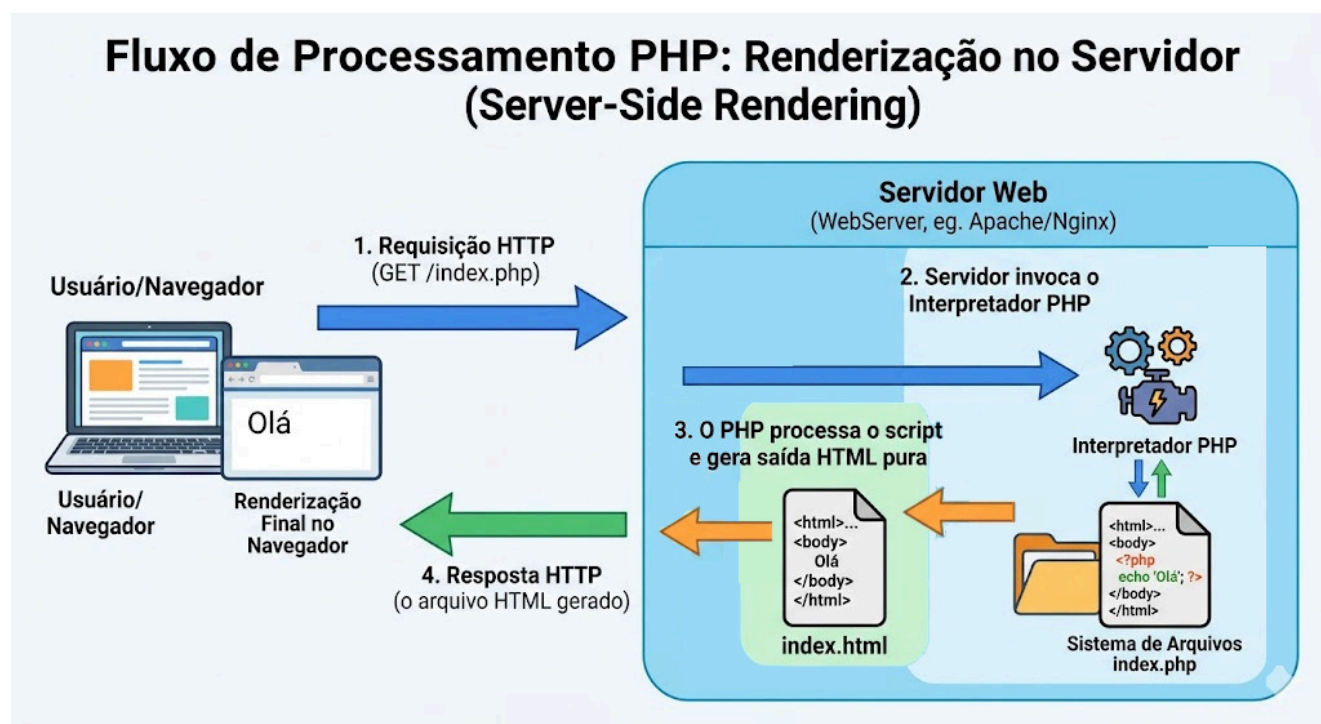
Starter-code

<https://gitlab.com/ds122-alexkutzke/ds122-php-example>

Linguagem PHP

Tags PHP

Quando o interpretador do PHP processa um arquivo, ele procura pelas tags de abertura e fechamento, `<?php` e `?>`, que delimitam onde o código PHP começa e termina. Tudo o que estiver fora dessas tags é ignorado pelo interpretador e repassado diretamente como texto plano (geralmente HTML) para a saída. Isso permite que o código PHP seja embutido em documentos HTML.



Existem apenas duas formas recomendadas e suportadas no PHP moderno para a abertura de tags:

1. **Tag padrão:** `<?php ... ?>` (Utilizada para blocos de código e lógica).
2. **Tag curta de impressão (Echo tag):** `<?= ... ?>` (Disponível por padrão, utilizada exclusivamente para imprimir um valor ou variável diretamente no HTML. É o equivalente exato a `<?php echo ... ?>`).

Nota: Historicamente, o PHP possuía outras formas de abertura, como tags estilo ASP (`<%`) ou tags curtas (`<?`). O uso destas formas foi removido da linguagem ou é estritamente desencorajado devido a problemas de portabilidade e conflito com a sintaxe XML.

Escapando o HTML (Mesclando PHP e HTML)

A característica de ignorar o que está fora das tags permite arquivos de conteúdo misto, fundamentais para a criação de *templates*.

```
<p>Isto vai ser ignorado pelo PHP e enviado diretamente ao navegador.</p>
<?php echo 'Enquanto isto vai ser processado no servidor.'; ?>
<p>Isto também vai ser ignorado pelo interpretador.</p>
```

Esta funcionalidade é particularmente útil em instruções de controle de fluxo (condicionais e laços de repetição). O interpretador do PHP determina o resultado da condição e decide qual bloco HTML será enviado ao cliente.

Exemplo 1: Sintaxe Alternativa para Estruturas de Controle

Para embutir código PHP em grandes blocos de HTML, a utilização da sintaxe alternativa (com dois pontos `:` e palavras-chave terminadoras como `endif;`, `endforeach;`) é a prática padrão e torna o código mais legível do que o uso de chaves `{}`.

```
<?php if ($usuarioLogado == true): ?>
    <p>Bem-vindo ao sistema! Este bloco HTML só será renderizado se a condição for verdadeira.</p>
<?php else: ?>
    <p>Por favor, faça o login para continuar.</p>
<?php endif; ?>
```

Nesse exemplo, o interpretador pulará os blocos HTML em que a condição não for satisfeita. Sair do modo de interpretação do PHP (fechando a tag `?>`) para escrever HTML puro é, na maioria dos casos, mais eficiente e limpo do que armazenar grandes blocos de texto HTML dentro de funções `echo` ou variáveis.

Separação de instruções e Omissão da Tag de Fechamento

O PHP exige que as instruções sejam terminadas com um ponto e vírgula (`;`).

A tag de fechamento de um bloco de código `?>` implica automaticamente no encerramento da instrução anterior, dispensando o ponto e vírgula na última linha antes do fechamento.

Regra Crítica de Boas Práticas: Se um arquivo contiver **apenas** código PHP (sem HTML misturado, como arquivos de classes, configurações ou funções), a tag de fechamento `?>` **deve ser omitida**. Isso previne a injeção acidental de espaços em branco ou quebras de linha invisíveis no final do arquivo, o que causa erros de envio de cabeçalhos HTTP (*Headers already sent*).

```
<?php
// Arquivo contendo apenas lógica PHP.
// A tag de fechamento ?> NÃO deve ser inserida ao final do arquivo.

echo 'Isto é um teste';
$variavel = 10;
```

Links importantes para a Sintaxe Básica

- [Tipos de Dados](#)
- [Variáveis](#)
- [Constantes](#)
- [Expressões](#)
- [Operadores](#)
- [Estruturas de Controle](#)
- [Funções](#)

Produção de conteúdo dinâmico

Um dos objetivos centrais da programação Web no servidor é produzir o conteúdo das páginas de maneira dinâmica, baseando-se em estruturas de dados (vetores, dicionários) e lógica de negócio.

O código a seguir demonstra a produção de uma lista não ordenada (`<ul>`) no HTML a partir de um *array* no PHP.

```

<?php
// É uma boa prática informar os tipos de dados que a função espera e retorna
function cria_item_lista(string|int $item): string {
    return "<li>{$item}</li>";
}

function cria_lista(array $itens): string {
    $result = "<ul>\n";
    foreach ($itens as $value) {
        $result .= cria_item_lista($value);
    }
    $result .= "</ul>\n";

    return $result;
}
?>

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Testes de PHP</title>
</head>
<body>

    <h2>Lista Gerada Dinamicamente:</h2>

    <?php
        // Utiliza-se a sintaxe moderna de arrays [] (colchetes)
        $meusItens = [1, 2, 4, 5, 6, "Texto Dinâmico"];

        // A tag de impressão curta <?= atua como um echo
    ?>

    <?= cria_lista($meusItens) ?>

</body>
</html>

```

O exemplo acima ilustra a intersecção entre a lógica de programação do lado do servidor (processamento do *array*) e a entrega de um documento formatado para o navegador do cliente.

Aula 12 - PHP - SuperGlobals, Formulários, Validação e Includes.

Bibliografia recomendada para o tema:

- [Documentação Oficial do PHP - Variáveis Pré-definidas](#)
- [w3schools - PHP SuperGlobals](#)
- [w3schools - PHP Forms](#)

Código de exemplo

<https://gitlab.com/ds122-alexkutzke/ds122-php-forms-example>

Arrays SuperGlobals

Ao iniciar um script PHP, o interpretador instancia automaticamente uma série de Arrays associativos globais, comumente denominados **SuperGlobals**. Essas variáveis estão disponíveis em qualquer escopo do código (dentro ou fora de funções) e carregam informações vitais sobre o ambiente do servidor, cabeçalhos HTTP e, principalmente, os dados enviados pelo usuário na requisição.

Os Arrays SuperGlobals mais utilizados são:

- `$_SERVER` : Informações sobre o ambiente do servidor, cabeçalhos HTTP, caminhos e localização de scripts.
- `$_POST` : Reúne dados enviados por um formulário utilizando o método HTTP POST (dados invisíveis na URL).
- `$_GET` : Reúne dados enviados através da URL (parâmetros de *Query String*) ou formulários com método HTTP GET.
- `$_REQUEST` : Um array que contém o conteúdo combinado de `$_GET`, `$_POST` e `$_COOKIE`. (Recomenda-se usar `$_GET` ou `$_POST` explicitamente por questões de segurança e clareza).
- `$_FILES` : Controla informações sobre arquivos enviados por Upload.
- `$_SESSION` : Informações sobre as sessões ativas do usuário.
- `$_COOKIE` : Cookies enviados pelo navegador do cliente.
- `$GLOBALS` : Acesso a todas as variáveis globais definidas no código (seu uso direto para alterar variáveis de fora de uma função é considerado má prática na programação moderna, devendo-se priorizar o retorno de funções e passagem de parâmetros).
- `$_ENV` : Variáveis de ambiente.

Exemplo de consulta ao array `$_SERVER` :

```
<?php
// Exibindo informações sobre o servidor e a requisição atual
echo "Caminho do script: " . htmlspecialchars($_SERVER['PHP_SELF']) . "<br>";
echo "Nome do servidor: " . $_SERVER['SERVER_NAME'] . "<br>";
echo "Host: " . $_SERVER['HTTP_HOST'] . "<br>";
echo "Navegador/User-Agent: " . $_SERVER['HTTP_USER_AGENT'] . "<br>";
?>
```

Atenção: O uso de `htmlspecialchars()` no `PHP_SELF` é fundamental para evitar ataques de injeção de código, como veremos na seção de XSS.

Formulários

A forma primária de coletar informações dos usuários em uma aplicação Web clássica é através de formulários HTML. Para que a comunicação entre o front-end e o back-end ocorra, o formulário precisa definir dois atributos cruciais:

1. **action** : A URL (recurso ou arquivo PHP) para onde os dados serão enviados.
2. **method** : O verbo HTTP utilizado para o envio (`GET` ou `POST`).

Além disso, é obrigatório que cada campo do formulário (`<input>` , `<select>` , `<textarea>`) possua o atributo **name** . O PHP utilizará o valor desse atributo como a “chave” no array SuperGlobal para resgatar o dado digitado.

Abaixo, um exemplo de um formulário de busca simples:

```
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="utf-8">
    <title>Teste de formulário</title>
  </head>
  <body>
    <form action="form_test.php" method="get">
      <label for="busca">Pesquisar:</label>
      <input type="text" id="busca" name="termo_busca">
      <button type="submit">Buscar</button>
    </form>
  </body>
</html>
```

Quando o usuário clicar em “Buscar”, o navegador fará uma requisição para `form_test.php?termo_busca=valor_digitado` .

Abaixo, o arquivo `form_test.php` que processa essa requisição:

```

<?php
// O operador ?? (Null Coalescing) verifica se o campo foi enviado.
// Se não foi, atribui uma string vazia, evitando erros de "Undefined array key".
$busca = $_GET["termo_busca"] ?? '';
?>

<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="utf-8">
    <title>Resultado da Busca</title>
  </head>
  <body>
    <h2>Você buscou por:</h2>
    <!-- O htmlspecialchars protege contra a execução acidental de tags HTML digitadas
pelo usuário -->
    <p><?= htmlspecialchars($busca) ?></p>
  </body>
</html>

```

Formulários com a mesma página (Self-submitting)

É muito comum que o formulário e o código PHP que o processa estejam no mesmo arquivo. Para isso, omitimos o atributo `action` ou apontamos para o próprio script. Também verificamos o método da requisição para saber se devemos apenas exibir o formulário ou também processar os dados.

```

<?php
$nome = '';
$mensagem = '';

// Verifica se o formulário foi submetido
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Resgata o valor. Se não existir, assume string vazia.
    $nome = $_POST['fname'] ?? '';

    if (empty(trim($nome))) {
        $mensagem = "O nome é obrigatório!";
    } else {
        $mensagem = "Olá, " . htmlspecialchars($nome) . "!";
    }
}
?>

<!DOCTYPE html>
<html>
<body>

<!-- htmlspecialchars impede que o PHP_SELF seja vetor de ataque -->
<form method="post" action="<?= htmlspecialchars($_SERVER['PHP_SELF']) ?>">
  <label for="fname">Nome:</label>
  <input type="text" id="fname" name="fname" value="<?= htmlspecialchars($nome) ?>">
  <button type="submit">Enviar</button>
</form>

<p><strong><?= $mensagem ?></strong></p>

</body>
</html>

```

Validação

Validação é o processo de verificar se os dados recebidos estão dentro das regras de negócio esperadas e se são seguros. Nunca se deve confiar nos dados enviados pelo usuário.

A validação ocorre em duas camadas:

1. **Front-end (Client-side):** Melhora a experiência do usuário, alertando erros imediatamente antes do envio, poupando processamento do servidor. Pode ser burlada facilmente.
2. **Back-end (Server-side):** É a validação definitiva e obrigatória. Garante a integridade e segurança do sistema, pois não pode ser manipulada pelo usuário.

Validação Front-end (Client-side)

A forma mais moderna e eficiente de fazer validação no front-end é utilizando os próprios atributos do **HTML5**, sem necessidade imediata de JavaScript.

- Atributos úteis: `required`, `minlength`, `maxlength`, `min`, `max`, `pattern` (Expressões Regulares), e os tipos semânticos (`type="email"`, `type="number"`).

Caso lógicas mais complexas sejam necessárias, utilizamos **JavaScript puro (Vanilla JS)** para interceptar o evento de submissão (`submit`):

```
<form id="form-test" action="processa.php" method="post">
  <input type="text" name="nome" id="nome_input">
  <span id="erro-nome" style="color: red;"></span>
  <button type="submit">Enviar</button>
</form>

<script>
  document.getElementById("form-test").addEventListener("submit", function(event) {
    const nomeInput = document.getElementById("nome_input").value;
    const erroSpan = document.getElementById("erro-nome");

    // Se estiver vazio, exibe o erro e cancela o envio
    if (nomeInput.trim() === "") {
      erroSpan.textContent = "O nome é obrigatório.";
      event.preventDefault(); // Impede o formulário de ser enviado ao servidor
    } else {
      erroSpan.textContent = "";
    }
  });
</script>
```

Validação Back-end (Server-side)

No PHP, verificamos se os dados existem, se estão no formato correto e os higienizamos (Sanitização).

Um fluxo clássico de sanitização básica manual envolve a remoção de espaços nas extremidades e a conversão de caracteres especiais HTML:


```

<?php
function limpa_dado($texto) {
    $texto = trim($texto); // Remove espaços no início e no final
    $texto = htmlspecialchars($texto, ENT_QUOTES, 'UTF-8'); // Converte caracteres como
    < e > para entidades HTML
    return $texto;
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nome = limpa_dado($_POST["nome"] ?? '');

    if (empty($nome)) {
        echo "O campo nome é inválido ou está vazio.";
    }
}
?>

```

Para validações avançadas e modernas, o PHP oferece a família de funções **filter_var**, que já vem com regras prontas para e-mails, URLs e números:

- Documentação: [filter_var](#)
-

Segurança: XSS e SQL Injection

A internet é um ambiente hostil. Formulários são a principal porta de entrada para ataques contra aplicações Web.

Cross-site Scripting (XSS)

Ocorre quando a aplicação recebe dados maliciosos contendo scripts (como JavaScript) e os exibe para outros usuários sem o devido tratamento.

Se um usuário preencher o campo “Nome” com: José <script>alert('Rubei seus cookies!');</script>

E a aplicação fizer apenas `echo $_POST['nome'];`, o navegador interpretará a tag <script> e executará o código.

Como prevenir: Utilize **SEMPRE** a função `htmlspecialchars()` antes de exibir qualquer dado gerado pelo usuário no HTML. Ela transforma < em < e > em >, fazendo o navegador tratar o conteúdo apenas como texto inofensivo.

SQL Injection (Injeção de SQL)

Consiste no envio de trechos de código SQL malicioso através dos formulários para manipular a estrutura da consulta (Query) feita ao Banco de Dados.

Se o código PHP for escrito assim: `$sql = "SELECT * FROM Usuarios WHERE id = " . $_POST['id'];`

O usuário pode digitar no campo ID: 105 OR 1=1 . A consulta final no servidor ficará: `SELECT * FROM Usuarios WHERE id = 105 OR 1=1;`

Como 1=1 é sempre verdadeiro, o banco retornará os dados de **todos** os usuários, burlando o sistema.

Como prevenir: No desenvolvimento back-end moderno, previne-se injeção de SQL utilizando a biblioteca **PDO** com as chamadas **Prepared Statements** (Consultas Preparadas), onde o dado enviado pelo usuário é tratado separadamente da instrução SQL.

Includes e Requires

É uma boa prática modularizar sistemas grandes separando o código em múltiplos arquivos menores (ex: um arquivo para o cabeçalho, um para o rodapé, outro para conexão com o banco).

O PHP possui quatro construções de linguagem para inclusão de arquivos:

- `include 'arquivo.php';` : Tenta incluir e avaliar o arquivo especificado. Se o arquivo não existir, o PHP emite um **Aviso (Warning)**, mas **continua a execução** do restante do script.
- `require 'arquivo.php';` : Tenta incluir o arquivo. Se o arquivo não existir, o PHP emite um **Erro Fatal (Fatal Error)** e **interrompe a execução** do script imediatamente. (Mais seguro para arquivos cruciais).
- `include_once` e `require_once` : Funcionam de forma idêntica aos seus pares, porém o PHP verificará se o arquivo já foi incluído anteriormente durante aquela execução. Se já foi, não o incluirá novamente, evitando redefinição de funções ou variáveis em cascata.

Exemplo de uso prático:

```
// arquivo: variaveis.php
<?php
$cor = 'verde';
$fruta = 'maçã';
?>

// arquivo: teste.php
<?php
// Tenta imprimir variáveis que ainda não existem
echo "Uma $fruta $cor. <br>";

// Inclui o arquivo que contém as variáveis
require 'variaveis.php';

// Agora as variáveis estão disponíveis no escopo
echo "Uma $fruta $cor."; // Saída: Uma maçã verde.
?>
```

Aulas 13 - PHP - Manipulação de bancos de dados

Bibliografia recomendada para o tema:

- [Documentação Oficial do PHP - Extensão Mysqli](#)
 - [w3schools - PHP+MySQL](#)
 - [PHP The Right Way - Bancos de Dados](#)
-

Acesso ao MySQL

Para acessar o banco de dados MySQL a partir de um script PHP, temos as seguintes opções:

1. Biblioteca (ou extensão) `mysqli` no formato procedural;
2. Biblioteca (ou extensão) `mysqli` no formato orientado a objetos;
3. Biblioteca `PDO` (PHP Data Objects).

Durante a disciplina, utilizaremos a **mysqli procedural**, pois não depende de conceitos de orientação a objetos — que serão estudados em disciplinas futuras. O foco aqui está na lógica de manipulação de dados.

Nota sobre PDO: O PDO é a recomendação moderna da comunidade PHP. Ele oferece suporte a **12 bancos de dados** diferentes (MySQL, PostgreSQL, SQLite, etc.) e utiliza *Prepared Statements* com *bind* de parâmetros nomeados, oferecendo maior segurança contra SQL Injection. Quando você avançar em Programação Orientada a Objetos, migrar de `mysqli` procedural para PDO será um passo natural. Uma introdução ao PDO é apresentada ao final desta aula.

Conexão com o Banco de Dados

Antes de qualquer operação, é necessário estabelecer uma conexão com o servidor MySQL. A função `mysqli_connect()` recebe os parâmetros de acesso e retorna um *link* de conexão.

Boas práticas:

- Armazene os dados de conexão (host, usuário, senha, nome do banco) em **variáveis** ou em um arquivo de configuração separado. Assim, se os dados mudarem, você altera em um único lugar.
- Utilize `mysqli_connect_error()` para verificar se a conexão foi bem-sucedida.

- É possível definir o charset da conexão com `mysqli_set_charset()` para `utf8mb4`, que oferece suporte completo a caracteres especiais e emojis.

```
<?php
// Dados de conexão – em um projeto real, guarde em arquivo separado (ex: config.php)
$host  = "localhost";
$user  = "appuser";
$pass  = "web1";
$dbname = "minha_aplicacao";

// Estabelece a conexão
$conn = mysqli_connect($host, $user, $pass, $dbname);

// Verifica se houve erro
if (mysqli_connect_error()) {
    die("Erro de conexão: " . mysqli_connect_error());
}

// Define o charset para utf8mb4 (suporte a acentos e emojis)
mysqli_set_charset($conn, "utf8mb4");

echo "Conectado com sucesso!";
?>
```

Em projetos maiores, isole a conexão em um arquivo `conexao.php` e utilize `require 'conexao.php';` nos demais scripts. Assim você evita repetir o código de conexão em cada página.

Criando um Banco de Dados

A criação do banco pode ser feita via phpMyAdmin (XAMPP) ou diretamente pelo terminal. Para criar via PHP, utilizamos `mysqli_query()` :

```
<?php
$host  = "localhost";
$user  = "appuser";
$pass  = "web1";

// Estabelece a conexão sem uma base de dados
$conn = mysqli_connect($host, $user, $pass);

// Verifica se houve erro
if (mysqli_connect_error()) {
    die("Erro de conexão: " . mysqli_connect_error());
}

$sql = "CREATE DATABASE IF NOT EXISTS minha_aplicacao CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci";

if (mysqli_query($conn, $sql)) {
    echo "Banco de dados criado com sucesso!";
} else {
    echo "Erro ao criar banco: " . mysqli_error($conn);
}

mysqli_close($conn);
?>
```

Criando um arquivo de credenciais

```
<?php
$host  = "mysql";
$user  = "usuario";
$pass  = "senha";
$dbname = "minha_aplicacao";
?>
```

Criando uma Tabela

Após selecionar o banco, criamos as tabelas necessárias. Abaixo, um exemplo de tabela `tarefas` que será usada nos exemplos seguintes:

```
<?php
require 'credentials.php';

$conn = mysqli_connect($host, $user, $pass, $dbname);

if (mysqli_connect_error()) {
    die("Erro de conexão: " . mysqli_connect_error());
}

$sql = "CREATE TABLE IF NOT EXISTS tarefas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    descricao TEXT,
    concluida TINYINT(1) DEFAULT 0,
    data_criacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4";

if (mysqli_query($conn, $sql)) {
    echo "Tabela 'tarefas' criada com sucesso!";
} else {
    echo "Erro ao criar tabela: " . mysqli_error($conn);
}

mysqli_close($conn);
```

CRUD — Operações Fundamentais

CRUD é o acrônimo para as quatro operações básicas de persistência de dados:

Operação	SQL	Descrição
Create	INSERT	Inserir um novo registro
Read	SELECT	Consultar registros
Update	UPDATE	Atualizar um registro
Delete	DELETE	Remover um registro

Criando arquivo de conexão genérico

```
<?php
require_once 'credentials.php';

// Estabelece a conexão
$conn = mysqli_connect($host, $user, $pass, $dbname);

// Verifica se houve erro
if (mysqli_connect_error()) {
    die("Erro de conexão: " . mysqli_connect_error());
}
?>
```

Create — Inserindo Dados (INSERT)

Para inserir registros, utiliza-se `mysqli_query()` com um comando `INSERT INTO`.

```
<?php
require 'conexao.php';

$titulo    = "Estudar PHP";
$descricao = "Revisar os conceitos de manipulação de arrays e strings";

// Monta a query – ATENÇÃO: esta forma é vulnerável a SQL Injection (veja seção de
segurança)
$sql = "INSERT INTO tarefas (titulo, descricao) VALUES ('$titulo', '$descricao')";

if (mysqli_query($conn, $sql)) {
    echo "Tarefa inserida com sucesso! ID: " . mysqli_insert_id($conn);
} else {
    echo "Erro ao inserir: " . mysqli_error($conn);
}

mysqli_close($conn);
?>
```

`mysqli_insert_id()` retorna o ID auto-incremento gerado na última inserção — útil quando a tabela possui chave primária automática.

Inserção com Prepared Statements (Forma Segura)

Prepared Statements (consultas preparadas) separam a estrutura da query dos valores fornecidos pelo usuário, **eliminando o risco de SQL Injection**.

```
<?php
require 'conexao.php';

$titulo    = "Estudar PHP";
$descricao = "Revisar os conceitos de manipulação de arrays e strings";

// 1. Prepara a query com placeholders ?
$stmt = mysqli_prepare($conn, "INSERT INTO tarefas (titulo, descricao) VALUES (?, ?)");

// 2. Associa os parâmetros – 's' = string, 'i' = integer, 'd' = double, 'b' = blob
mysqli_stmt_bind_param($stmt, "ss", $titulo, $descricao);

// 3. Executa
if (mysqli_stmt_execute($stmt)) {
    echo "Tarefa inserida com sucesso! ID: " . mysqli_stmt_insert_id($stmt);
} else {
    echo "Erro ao inserir: " . mysqli_stmt_error($stmt);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Sempre que os valores da query vierem de fontes externas (formulários, URLs, APIs), utilize

Prepared Statements. É a principal defesa contra SQL Injection.

Read — Consultando Dados (SELECT)

Para consultar registros, executa-se um `SELECT` e recupera-se os resultados com `mysqli_fetch_assoc()`.

Listar todos os registros

```
<?php
require 'conexao.php';

$sql = "SELECT id, titulo, descricao, concluida, data_criacao FROM tarefas ORDER BY
data_criacao DESC";
$result = mysqli_query($conn, $sql);

// Verifica se há registros
if (mysqli_num_rows($result) > 0) {
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<strong>#{$row['id']} – {$row['titulo']}</strong><br>";
        echo "{$row['descricao']}<br>";
        echo "Concluída: " . ($row['concluida'] ? 'Sim' : 'Não') . "<br>";
        echo "<hr>";
    }
} else {
    echo "Nenhuma tarefa encontrada.";
}

mysqli_free_result($result);
mysqli_close($conn);
?>
```

Exibindo os dados em uma tabela HTML

É muito comum combinar PHP e HTML para exibir dados em formato tabular:


```

<?php require 'conexao.php'; ?>

<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Minhas Tarefas</title>
    <style>
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
        th { background-color: #f0f0f0; }
    </style>
</head>
<body>
    <h1>Lista de Tarefas</h1>

    <table>
        <tr>
            <th>ID</th>
            <th>Título</th>
            <th>Descrição</th>
            <th>Concluída</th>
            <th>Data</th>
        </tr>

        <?php
            $sql = "SELECT * FROM tarefas ORDER BY id ASC";
            $result = mysqli_query($conn, $sql);

            if (mysqli_num_rows($result) > 0):
                while ($row = mysqli_fetch_assoc($result)):
                    ?>
                    <tr>
                        <td><?= $row['id'] ?></td>
                        <td><?= htmlspecialchars($row['titulo']) ?></td>
                        <td><?= htmlspecialchars($row['descricao']) ?></td>
                        <td><?= $row['concluida'] ? '✅' : '❌' ?></td>
                        <td><?= $row['data_criacao'] ?></td>
                    </tr>
                <?php
                    endwhile;
                else:
                    ?>
                    <tr><td colspan="5">Nenhuma tarefa cadastrada.</td></tr>
                <?php endif; ?>
            </table>

            <?php mysqli_close($conn); ?>
        </body>
</html>

```

Consultar um único registro por ID

```
<?php
require 'conexao.php';

$id = $_GET['id'] ?? 0;

// Prepared Statement para busca por ID
$stmt = mysqli_prepare($conn, "SELECT * FROM tarefas WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);

if ($row = mysqli_fetch_assoc($result)) {
    echo "<h2>{$row['titulo']}</h2>";
    echo "<p>{$row['descricao']}</p>";
} else {
    echo "Tarefa não encontrada.";
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Limit e Paginação

Para limitar o número de registros retornados, utiliza-se a cláusula `LIMIT` :

```
$sql = "SELECT * FROM tarefas ORDER BY data_criacao DESC LIMIT 5";
```

Para paginação, combina-se `LIMIT` com `OFFSET` :

```
$pagina      = $_GET['pagina'] ?? 1;
$por_pagina  = 5;
$offset      = ($pagina - 1) * $por_pagina;

$sql = "SELECT * FROM tarefas ORDER BY id ASC LIMIT $por_pagina OFFSET $offset";
```

Update — Atualizando Dados (UPDATE)

Assim como nas inserções, recomenda-se o uso de Prepared Statements para garantir segurança:

```
<?php
require 'conexao.php';

$id          = 1;
$titulo     = "Estudar PHP – Revisado";
$concluida  = 1; // true

$stmt = mysqli_prepare($conn, "UPDATE tarefas SET titulo = ?, concluida = ? WHERE id = ?");
mysqli_stmt_bind_param($stmt, "sii", $titulo, $concluida, $id);

if (mysqli_stmt_execute($stmt)) {
    // mysqli_stmt_affected_rows retorna o número de linhas afetadas
    if (mysqli_stmt_affected_rows($stmt) > 0) {
        echo "Tarefa atualizada com sucesso!";
    } else {
        echo "Nenhuma tarefa foi alterada (ID não encontrado ou dados iguais).";
    }
} else {
    echo "Erro ao atualizar: " . mysqli_stmt_error($stmt);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Delete — Removendo Dados (DELETE)

A exclusão também deve ser feita com Prepared Statements, especialmente quando o identificador vem do usuário (ex: via URL):

```
<?php
require 'conexao.php';

// Em uma aplicação real, o ID viria de $_GET ou $_POST
$id = $_GET['id'] ?? 0;

$stmt = mysqli_prepare($conn, "DELETE FROM tarefas WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);

if (mysqli_stmt_execute($stmt)) {
    if (mysqli_stmt_affected_rows($stmt) > 0) {
        echo "Tarefa removida com sucesso!";
    } else {
        echo "Nenhuma tarefa encontrada com esse ID.";
    }
} else {
    echo "Erro ao remover: " . mysqli_stmt_error($stmt);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

Sempre lembre-se de adicionar uma cláusula `WHERE` ao comando `DELETE`.

SQL Injection — O Perigo das Queries Não Tratadas

Na aula 12, vimos que SQL Injection consiste no envio de trechos de SQL malicioso através de campos de formulário ou parâmetros de URL.

Exemplo de código vulnerável:

```
// NUNCA FAÇA ISSO em produção:
$id = $_GET['id'];
$sql = "SELECT * FROM tarefas WHERE id = $id";
$result = mysqli_query($conn, $sql);
```

Se o usuário acessar `?id=1 OR 1=1`, a query será:

```
SELECT * FROM tarefas WHERE id = 1 OR 1=1
```

Como `1=1` é sempre verdadeiro, **todos os registros** serão retornados.

Solução — Prepared Statements:

```
$stmt = mysqli_prepare($conn, "SELECT * FROM tarefas WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);
mysqli_stmt_execute($stmt);
```

Com Prepared Statements, o valor `1 OR 1=1` é tratado **apenas como dado**, nunca como parte do comando SQL. O banco entende que é uma string, não uma condição lógica.

Tipo	Caractere	Exemplo de uso
String	s	Títulos, nomes, textos
Integer	i	IDs, quantidades, booleanos
Double	d	Preços, valores decimais
Blob	b	Arquivos, imagens

PDO (PHP Data Objects)

Esta seção é informativa. O PDO utiliza Programação Orientada a Objetos e será abordado com mais profundidade quando você tiver essa base. O objetivo aqui é apresentar a alternativa para que você conheça o caminho recomendado pela comunidade PHP.

Por que o PDO é recomendado?

1. **Portabilidade:** o mesmo código funciona com MySQL, PostgreSQL, SQLite e outros 9 bancos.
2. **Named Parameters:** os placeholders têm nomes (`:nome`), deixando o código mais legível que `?`.
3. **Fetch Modes flexíveis:** é possível buscar dados como array associativo, objeto, ou instância de classe.

4. **Tratamento de exceções:** erros são reportados como `PDOException`, que podem ser capturadas com `try/catch`.

Comparação: mysqli procedural vs. PDO

mysqli procedural — Prepared Statement:

```
$stmt = mysqli_prepare($conn, "INSERT INTO tarefas (titulo, descricao) VALUES (?, ?)");
mysqli_stmt_bind_param($stmt, "ss", $titulo, $descricao);
mysqli_stmt_execute($stmt);
```

PDO — Prepared Statement com named parameters:

```
$pdo = new PDO("mysql:host=localhost;dbname=minha_aplicacao;charset=utf8mb4",
"appuser", "web1");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $pdo->prepare("INSERT INTO tarefas (titulo, descricao) VALUES (:titulo, :descricao)");
$stmt->execute([
    ':titulo' => $titulo,
    ':descricao' => $descricao,
]);
```

Instalação do MySQL

Windows (XAMPP)

A maioria dos alunos utiliza o **XAMPP**, que já inclui o **MariaDB** (compatível com MySQL) e a extensão PHP necessários. Não é preciso instalar nada adicional.

- O gerenciamento do banco pode ser feito pelo **phpMyAdmin**, disponível em <http://localhost/phpmyadmin>.
- O XAMPP inicia o MySQL/MariaDB automaticamente pelo painel de controle.

Linux (Debian/Ubuntu)

Para instalar o MySQL e a extensão do PHP no Ubuntu, execute:

```
sudo apt-get install mysql-server php-mysql
```

O pacote `php-mysql` instala automaticamente a extensão compatível com sua versão do PHP. Para descobrir sua versão, execute `php -v`.

Após a instalação, execute o assistente de segurança:

```
sudo mysql_secure_installation
```

Responda **“yes”** para todas as questões, **exceto** para a primeira (alteração da senha de root), caso queira manter a senha já configurada.

Reinicie o servidor web:

```
sudo service apache2 restart
```

Para testar a instalação, acesse o terminal do MySQL:

```
sudo mysql -uroot -p
```

Criando um usuário para a aplicação

Por boas práticas, não se utiliza o usuário `root` nas aplicações. Crie um usuário específico:

```
CREATE USER 'appuser'@'localhost' IDENTIFIED WITH mysql_native_password BY  
'sua_senha_aqui';  
GRANT ALL PRIVILEGES ON *.* TO 'appuser'@'localhost';  
FLUSH PRIVILEGES;
```

No lugar de `'sua_senha_aqui'`, defina uma senha de sua preferência. Para o ambiente de desenvolvimento local, pode-se utilizar uma senha simples como `web1`.

Nota: o método `mysql_native_password` é adequado para ambientes de desenvolvimento. Em produção, o MySQL 8+ utiliza `caching_sha2_password`, que é mais seguro.

Aulas 14 - PHP - Cookies, Sessions e Login

Bibliografia recomendada para o tema:

- [Documentação Oficial do PHP — Cookies](#)
 - [Documentação Oficial do PHP — Sessões](#)
 - [Sobre cookies no protocolo HTTP \(Wikipedia\)](#)
 - [w3schools — PHP Cookies](#)
 - [w3schools — PHP Sessions](#)
-

Por que precisamos de Cookies e Sessões?

O protocolo HTTP é **stateless** (sem estado): cada requisição é independente — o servidor não sabe se duas requisições consecutivas vieram do mesmo navegador. Isso funciona bem para páginas estáticas, mas não para aplicações que precisam lembrar quem é o usuário.

Cookies e Sessões resolvem esse problema, permitindo que o servidor **mantenha estado** entre requisições:

- **Cookies:** pequenos arquivos de texto armazenados no **navegador** do cliente.
 - **Sessões:** dados armazenados no **servidor**, indexados por um identificador único (Session ID) que trafega como cookie.
-

Cookies no Protocolo HTTP

Do site [Wikipedia — Cookie HTTP](#):

Um cookie é um pequeno pacote de dados enviados de um website para o navegador do usuário quando o usuário visita o site. Cada vez que o usuário visita o site novamente, o navegador envia o cookie de volta para o servidor para notificar atividades prévias do usuário. Os cookies foram designados para ser um mecanismo confiável para que sites se lembrem de informações da atividade do usuário, como senhas gravadas, itens adicionados no carrinho de compras em uma loja online, links que foram clicados anteriormente, entre outros.

Cookies no PHP — `setcookie()`

A criação de um cookie em PHP utiliza a função `setcookie()`, que recebe os seguintes parâmetros:

```

setcookie(
    string $name,          // Nome do cookie (obrigatório)
    string $value,         // Valor a armazenar
    int $expires,          // Timestamp de expiração (padrão: 0 = expira ao fechar o
navegador)
    string $path,          // Caminho no site onde o cookie é válido (padrão: diretório
atual)
    string $domain,        // Domínio onde o cookie é válido
    bool $secure,          // Se true, só envia por HTTPS
    bool $httponly         // Se true, inacessível via JavaScript (protege contra XSS)
);

```

Apenas o parâmetro `$name` é obrigatório.

Criando um cookie

```

<?php
// Cookie "usuario" com valor "João", válido por 30 dias, disponível em todo o site
setcookie("usuario", "João", time() + (86400 * 30), "/");
?>

```

Importante: `setcookie()` deve ser chamada **antes de qualquer saída HTML**, pois o cookie é enviado nos cabeçalhos da resposta HTTP.

Lendo um cookie — `$_COOKIE`

O array superglobal `$_COOKIE` contém os cookies enviados pelo navegador na requisição atual.

Atenção: um cookie definido com `setcookie()` **não estará imediatamente disponível** em `$_COOKIE` na mesma requisição — ele só aparecerá na **próxima** requisição, quando o navegador o reenviar.

```

<?php
// Página 1: definir_cookie.php
setcookie("usuario", "João", time() + (86400 * 30), "/");
?>
<!DOCTYPE html>
<html><body>
    <p>Cookie definido! <a href="ler_cookie.php">Ver cookie</a></p>
</body></html>

```

```

<?php
// Página 2: ler_cookie.php — acessada APÓS definir o cookie
?>
<!DOCTYPE html>
<html><body>
<?php if (isset($_COOKIE["usuario"])): ?>
    <p>Bem-vindo de volta, <?= htmlspecialchars($_COOKIE["usuario"]) ?>!</p>
<?php else: ?>
    <p>Cookie não encontrado.</p>
<?php endif; ?>
</body></html>

```


Alterando e removendo um cookie

Para **alterar** o valor, basta chamar `setcookie()` novamente com o mesmo nome e novo valor.

Para **remover**, defina uma data de expiração no passado:

```
// Remove o cookie "usuario"
setcookie("usuario", "", time() - 3600, "/");
```

Parâmetros de segurança

Parâmetro	O que faz
<code>secure</code>	O cookie só é enviado em conexões HTTPS. Essencial em produção.
<code>httponly</code>	Impede que JavaScript acesse o cookie (<code>document.cookie</code>), mitigando roubo via XSS.
<code>SameSite</code>	Controla se o cookie é enviado em requisições cross-site. Use <code>Lax</code> (padrão seguro) ou <code>Strict</code> .

Exemplo de cookie seguro:

```
// Cookie com flags de segurança ativas (PHP 7.3+ para SameSite no array de opções)
setcookie("usuario", "João", [
    "expires" => time() + (86400 * 30),
    "path"    => "/",
    "secure"  => true,    // Apenas HTTPS
    "httponly" => true,   // Inacessível via JavaScript
    "samesite" => "Lax",  // Proteção contra CSRF
]);
```

Para ambiente de desenvolvimento local (`localhost` sem HTTPS), mantenha `secure` como `false`.

Sessões

Enquanto cookies armazenam dados no **navegador** (visíveis e modificáveis pelo usuário), sessões armazenam dados no **servidor**. O navegador guarda apenas um identificador (Session ID) em um cookie.

Fluxo de uma sessão

1. O PHP gera um **Session ID** único.
2. O Session ID é enviado ao navegador como cookie (`PHPSESSID`).
3. Os dados da sessão ficam armazenados no servidor (arquivo ou banco).
4. A cada requisição, o navegador reenvia o Session ID, e o PHP recupera os dados correspondentes.

Iniciando uma sessão — session_start()

```
<?php
// Sempre no topo do arquivo, antes de qualquer saída HTML
session_start();
?>
```

Use `session_status()` para verificar se já existe uma sessão ativa e evitar chamar `session_start()` duas vezes:

```
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
```

Gravando e lendo dados da sessão — \$_SESSION

O array superglobal `$_SESSION` funciona como um array associativo comum — você grava e lê valores livremente:

```
<?php
session_start();

// Gravando dados (ex: após login bem-sucedido)
$_SESSION["usuario_id"]    = 42;
$_SESSION["usuario_nome"]  = "Maria Silva";
$_SESSION["logado_em"]     = time();
?>
```

```
<?php
// Lendo dados em qualquer outra página
session_start();

if (isset($_SESSION["usuario_id"])) {
    echo "Bem-vinda, " . htmlspecialchars($_SESSION["usuario_nome"]) . "!";
} else {
    echo "Você não está logado.";
}
?>
```

Encerrando uma sessão (logout)

```
<?php
session_start();

// 1. Limpa todas as variáveis da sessão
$_SESSION = [];

// 2. Remove o cookie de sessão do navegador
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time() - 3600,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// 3. Destrói a sessão no servidor
session_destroy();

// 4. Redireciona para a página de login
header("Location: login.php");
exit;
?>
```

Segurança: session_regenerate_id()

Após o login, é boa prática **regenerar o Session ID** para evitar *Session Fixation* (quando um atacante força o uso de um ID conhecido):

```
session_regenerate_id(true); // true = apaga o arquivo de sessão antigo
```

Sistema de Login com Sessões e Banco de Dados

Abaixo, um exemplo completo de sistema de login que utiliza **mysqli + Prepared Statements** e sessões PHP.

Estrutura de arquivos

/	
— config.php	# Dados de conexão
— conexao.php	# Conexão com MySQL
— criar_tabela.php	# Script para criar a tabela de usuários
— cadastrar.php	# Cadastro de usuário (com hash de senha)
— login.php	# Formulário + processamento de login
— dashboard.php	# Página protegida (só acessa logado)
— logout.php	# Encerra a sessão

config.php — Dados de conexão

```
<?php
// Arquivo de configuração – não commitar credenciais reais em repositórios públicos
define("DB_HOST", "localhost");
define("DB_USER", "appuser");
define("DB_PASS", "web1");
define("DB_NAME", "minha_aplicacao");
?>
```

conexao.php — Conexão com o banco

```
<?php
require_once __DIR__ . "/config.php";

$conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if (mysqli_connect_error()) {
    die("Erro de conexão: " . mysqli_connect_error());
}

mysqli_set_charset($conn, "utf8mb4");
?>
```

criar_tabela.php — Tabela de usuários

```
<?php
require_once __DIR__ . "/conexao.php";

$sql = "CREATE TABLE IF NOT EXISTS usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4";

mysqli_query($conn, $sql);
echo "Tabela 'usuarios' pronta!";
mysqli_close($conn);
?>
```

cadastrar.php — Cadastro com senha segura

```
<?php
require_once __DIR__ . "/conexao.php";

$nome = "Admin";
$email = "admin@exemplo.com";
$senha_plana = "123456";

// NUNCA armazene a senha em texto plano – use password_hash()
$senha_hash = password_hash($senha_plana, PASSWORD_DEFAULT);

$stmt = mysqli_prepare($conn, "INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?)");
mysqli_stmt_bind_param($stmt, "sss", $nome, $email, $senha_hash);

if (mysqli_stmt_execute($stmt)) {
    echo "Usuário cadastrado com sucesso!";
} else {
    echo "Erro: " . mysqli_stmt_error($stmt);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);
?>
```

password_hash() : função nativa do PHP que gera um hash criptográfico seguro usando o algoritmo bcrypt (ou Argon2, se disponível). **Nunca** utilize `md5()` ou `sha1()` para senhas — são facilmente quebráveis. Para verificar a senha, utilize `password_verify()`.

login.php — Formulário e processamento

```
<?php
session_start();

// Se já estiver logado, redireciona
if (isset($_SESSION["usuario_id"])) {
    header("Location: dashboard.php");
    exit;
}

$erro = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    require_once __DIR__ . "/conexao.php";

    $email = $_POST["email"] ?? "";
    $senha = $_POST["senha"] ?? "";

    if (empty($email) || empty($senha)) {
        $erro = "Preencha todos os campos.";
    } else {
        // Prepared Statement para buscar o usuário pelo email
        $stmt = mysqli_prepare($conn, "SELECT id, nome, senha FROM usuarios WHERE
email = ?");
        mysqli_stmt_bind_param($stmt, "s", $email);
        mysqli_stmt_execute($stmt);
        $result = mysqli_stmt_get_result($stmt);

        if ($usuario = mysqli_fetch_assoc($result)) {
            // Verifica a senha com password_verify()
            if (password_verify($senha, $usuario["senha"])) {
                // Login bem-sucedido → regenera ID e grava sessão
                session_regenerate_id(true);
                $_SESSION["usuario_id"] = $usuario["id"];
                $_SESSION["usuario_nome"] = $usuario["nome"];

                mysqli_stmt_close($stmt);
                mysqli_close($conn);

                header("Location: dashboard.php");
                exit;
            }
        }

        // Login falhou (email não encontrado OU senha incorreta)
        $erro = "Email ou senha inválidos.";
        mysqli_stmt_close($stmt);
        mysqli_close($conn);
    }
}

?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Login</title>
</head>
<body>
    <h1>Login</h1>

    <?php if ($erro): ?>
        <p style="color: red;"><?= htmlspecialchars($erro) ?></p>
    <?php endif; ?>
```

```

<form method="post" action="login.php">
  <label for="email">Email:</label><br>
  <input type="email" id="email" name="email" required><br><br>

  <label for="senha">Senha:</label><br>
  <input type="password" id="senha" name="senha" required><br><br>

  <button type="submit">Entrar</button>
</form>
</body>
</html>

```

dashboard.php — Página protegida

```

<?php
session_start();

// Proteção: se não estiver logado, redireciona para o login
if (!isset($_SESSION["usuario_id"])) {
  header("Location: login.php");
  exit;
}
?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Painel</title>
</head>
<body>
  <h1>Painel do Usuário</h1>
  <p>Bem-vindo(a), <strong><?= htmlspecialchars($_SESSION["usuario_nome"]) ?>
</strong>!</p>
  <p>Esta página só pode ser acessada por usuários autenticados.</p>

  <p><a href="logout.php">Sair</a></p>
</body>
</html>

```

logout.php — Encerrando a sessão

```
<?php
session_start();

// Limpa variáveis da sessão
$_SESSION = [];

// Remove o cookie PHPSESSID do navegador
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time() - 3600,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

// Destrói a sessão no servidor
session_destroy();

// Redireciona para o login
header("Location: login.php");
exit;
?>
```

Fluxo completo

1. O usuário acessa `login.php` e preenche email/senha.
 2. O PHP busca o email no banco com **Prepared Statement** (protegido contra SQL Injection).
 3. `password_verify()` compara a senha digitada com o hash armazenado.
 4. Se válido: `session_regenerate_id()` + grava dados em `$_SESSION` + redireciona para `dashboard.php`.
 5. Toda página protegida verifica `$_SESSION["usuario_id"]` e redireciona para `login.php` se ausente.
 6. O logout limpa `$_SESSION`, remove o cookie e destrói a sessão.
-

Referência completa do exemplo

O código completo deste sistema de login está disponível em:

<https://gitlab.com/ds122-alexkutzke/ds122-login-app>

Especificação de Trabalho Prático 01/2026

O trabalho prático envolve a criação de uma aplicação WEB completa. Ou seja, que inclua a implementação de front-end, back-end e que possua integração com um banco de dados.

Tema

A aplicação deve implementar um jogo de digitação utilizando Javascript e utilizar PHP para armazenar e exibir quadros de pontuação.

O funcionamento é o seguinte:

1. O usuário deve se registrar e se autenticar para acessar o sistema;
2. Uma vez autenticado, o usuário pode jogar partidas de um jogo de digitação;
3. A cada partida, o usuário acumula pontos, exibidos pelo sistema.
4. O usuário pode acessar seu histórico de partidas (e pontuação), bem como diferentes quadros de pontuação (pelo menos geral e ligas)

O jogo de digitação a ser implementado é livre, desde que envolva o princípio básico de digitação correta de palavras. Os jogos typing.com e [ztype](https://ztype.com) são bons exemplos desse princípio.

O sistema deve disponibilizar a inscrição do usuário em ligas. Ligas são um conjunto de usuários que competem entre si. O usuário pode criar e se cadastrar em ligas. Para o cadastro do usuário em uma liga é necessário uma palavra-chave, definida pelo criador da liga.

A pontuação da liga deve ser exibida de duas formas:

1. pontuação desde a criação da liga; e
2. pontuação semanal.

Além da pontuação em suas respectivas ligas, o usuário também pode verificar sua pontuação geral, envolvendo todos os jogadores. Esse quadro também deve apresentar a pontuação desde a criação do sistema e pontuação semanal.

A qualquer momento, o usuário pode acessar um relatório com os dados de todas as partidas jogadas, com suas respectivas pontuações.

Requisitos

A aplicação desenvolvida deve atender os seguintes requisitos:

- **Front-end:**
 - Uso de HTML5, CSS3 e JS;
 - Interface amigável; ;
 - Validação de campos de formulário;
 - Implementação do Jogo de digitação completamente em JS;
- **Back-end;**
 - Integração com um banco de dados;
 - Sistema de autenticação/autorização de usuário(s) salvo(s) em banco de dados;
 - Validação de campos de formulário e outras informações recebidas.

Ambiente de Desenvolvimento

- O sistema deve ser desenvolvido utilizando **apenas** os recursos demonstrados na disciplina DS122 (PHP, Javascript (jQuery), HTML5, CSS3 e algum banco de dados);
 - É permitido o uso de frameworks *front-end*, como Bootstrap e W3.CSS;
 - **Não** é permitido o uso de frameworks *back-end*.

Entrega

Datas de entrega e defesa na UFPR Virtual.

O trabalho pode ser feito em **grupos de 2 até 4 alunos**.

O código deve ser entregue através da UFPR Virtual, por meio de link para repositório git.

O trabalho deverá ser defendido através de uma demonstração do seu funcionamento e explicação do código. A defesa é realizada apenas para o professor, não para a turma.

Documentação

O repositório deverá conter um arquivo chamado `README.md` com a descrição do sistema e de seu funcionamento. Deve-se utilizar a sintaxe correta da linguagem **Markdown** nesse documento (para saber mais, consulte: <https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet>).

Critério para avaliação

Os critérios para avaliação serão os seguintes:

- **Defesa e conhecimento do código** [50 pontos]:
- **Funcionalidades e implementação** [50 pontos]:
 - Qualidade da interface do usuário [10 ponto];
 - Funcionamento do Jogo de digitação [20 ponto];
 - Funcionamento da aplicação back-end [20 pontos];

Atenção: em nenhuma hipótese serão aceitos trabalhos com qualquer traço de plágio. A identificação de plágio implica em nota zero a todos os integrantes do grupo.

Uso de IA Generativa

O uso de IA Generativa é permitido segundo algumas regras, descritas abaixo, e perante entrega de relatório das interações realizadas.

O repositório com o código do trabalho deverá conter um arquivo chamado `AI_USAGE_LOG.md` com o seguinte conteúdo:

Relatório de Uso de Inteligência Artificial Generativa

Este documento registra todas as interações significativas com ferramentas de IA generativa (como Gemini, ChatGPT, Copilot, etc.) durante o desenvolvimento deste projeto. O objetivo é promover o uso ético e transparente da IA como ferramenta de apoio, e não como substituta para a compreensão dos conceitos fundamentais.

Política de Uso

O uso de IA foi permitido para as seguintes finalidades:

- Geração de ideias e brainstorming de algoritmos.
- Explicação de conceitos complexos.
- Geração de código boilerplate (ex: estrutura de classes, leitura de arquivos).
- Sugestões de refatoração e otimização de código.
- Debugging e identificação de causas de erros.
- Geração de casos de teste.

É proibido submeter código gerado por IA sem compreendê-lo completamente e sem adaptá-lo ao projeto. Todo trecho de código influenciado pela IA deve ser referenciado neste log.

Registro de Interações

Copie e preencha o template abaixo para cada interação relevante.

Interação 1

- ****Data:**** 20/10/2025
- ****Etapa do Projeto:**** 1 - Compressão de Arquivos
- ****Ferramenta de IA Utilizada:**** Gemini Advanced
- ****Objetivo da Consulta:**** Eu estava com dificuldades para entender como gerenciar o dicionário do algoritmo LZW quando ele atinge o tamanho máximo. Precisava de uma estratégia para lidar com isso.

- ****Prompt(s) Utilizado(s):****

1. "No algoritmo de compressão LZW, o que acontece quando o dicionário atinge o tamanho máximo? Quais são as estratégias mais comuns para lidar com isso?"
2. "Pode me dar um exemplo em Python de como implementar a estratégia de 'resetar o dicionário' no LZW?"

- ****Resumo da Resposta da IA:****

A IA explicou três estratégias: 1) parar de adicionar novas entradas, 2) resetar o dicionário para o estado inicial, e 3) usar uma política de descarte, como LRU (Least Recently Used), que é mais complexa. A IA forneceu um pseudocódigo para a estratégia de reset, que parecia a mais simples e eficaz para este projeto.

- ****Análise e Aplicação:****

A resposta da IA foi extremamente útil para clarear as opções. Optei por implementar a estratégia de resetar o dicionário. O código fornecido pela IA não foi usado diretamente, pois estava muito simplificado e não se encaixava na minha arquitetura de classes. No entanto, a lógica de verificar o tamanho do dicionário e invocar uma função `reset\_dictionary()` foi a base para a minha implementação. Isso me poupou tempo de pesquisa em artigos e livros.

- ****Referência no Código:****

A lógica inspirada por esta interação foi implementada no arquivo `compressor/lzw.py`, especificamente na função `compress()`, por volta da linha 85.

Interação 2

- ****Data:**** ...
- ****Etapa do Projeto:**** ...

- ****Ferramenta de IA Utilizada:**** ...
- ****Objetivo da Consulta:**** ...
- ****Prompt(s) Utilizado(s):**** ...
- ****Resumo da Resposta da IA:**** ...
- ****Análise e Aplicação:**** ...
- ****Referência no Código:**** ...
